PURBANCHAL UNIVERSITY SCHOOL OF ENGINEERING (PUSOE)

BACHELOR IN COMPUTER ENGINEERING (BE.COM)

A PROJECT REPORT ON

"**smartRecruit – Resume Analyzer using SVC**"

(Course code: BEG479CO)

Submitted By:

Adesh Jain (381573)

Aikik Dahal (381574)

Abhishek Chaudhary(381572)

Submitted To:

P.U. School of Engineering (PUSOE)

Submitted Date: **2082-04-23**

PURBANCHAL UNIVERSITY SCHOOL OF ENGINEERING
Biratnagar, Nepal
Website: https://puse.edu.np/

Date: 2082-04-23

Certificate of Approval

This is to certify that the project entitled " **smartRecruit – Resume Analyzer using SVC** " is being submitted by:

Name of the Students	Symbol No.
Adesh Jain	**381573**
Aikik Dahal	**381574**
Abhishek Chaudhary	**381572**

In partial fulfillment of the requirements for the degree of Bachelor of Engineering in Computer Engineering (BE Computer) from Purbanchal University School of Engineering (PUSOE), Kanya Marg, Biratnagar, Nepal, this project presents original work that has not been submitted to any other university or institution for the award of any degree or diploma.

…… ……

Internal Supervisor **External Supervisor**

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to all those who contributed to the successful completion of our 8th semester project titled **"smartRecruit – Resume Analyzer using SVC"**.

We extend our sincere thanks to our respected internal supervisor, **Er. Santosh Sah**, whose consistent guidance, valuable feedback, and expert mentorship greatly enhanced the quality of our project. His encouragement and insights were instrumental in helping us incorporate new features such as resume-job matching, chatbot integration, and portfolio generation.

We also acknowledge the support and teachings of all faculty members of the **Department of Computer Engineering, Purbanchal University School of Engineering (PUSOE)**, which provided us with the theoretical and practical foundation necessary for this work.

We are grateful to our friends and classmates for their continuous support, suggestions, and cooperation during the development phase.

Finally, we thank our families for their unwavering motivation, patience, and support throughout our academic journey.

This project is the result of true teamwork and shared learning among the following group members:

- **Abhishek Chaudhary** (381572)
- **Adesh Jain** (381573)
- **Aikik Dahal** (381574)

ABSTRACT

In today's fast-paced job market, the process of reviewing and shortlisting resumes manually is both time-consuming and prone to bias. To address these challenges, **smartRecruit – Resume Analyzer using SVC** has been enhanced into a comprehensive AI-powered recruitment assistant capable of automating resume analysis, intelligent job matching, personalized feedback, and interactive support.

The updated system goes beyond traditional resume screening by incorporating advanced features such as a Retrieval-Augmented Generation (RAG) chatbot, which helps job seekers interactively understand their resume strengths and job fit. The platform uses **Natural** Language Processing (NLP) to extract key information like skills, education, and contact details, and applies Support Vector Classifier (SVC) and K-Nearest Neighbors (KNN) for job categorization and recommendation. Similarity between resumes and job descriptions is measured using TF-IDF vectorization and cosine similarity, enabling precise candidate-job matching.

New additions include a portfolio generation module that transforms user data into a professional web-based resume, interview question PDF generator powered by OpenAI, and course recommendations via the Microsoft Learn API based on skill gaps. The system also supports resume matching in batch mode, allowing HR managers to upload multiple resumes and instantly rank them against job descriptions.

Built with Flask**,** SQLite**,** and OpenAI APIs, smartRecruit offers a role-based interface for job seekers, HR professionals, and admins, making it scalable, secure, and user-friendly. This upgraded version represents a significant step toward intelligent, fair, and efficient hiring through automation and real-time AI assistance.

Table of Contents

LIST OF FIGURES

LIST OF ABBREVIATIONS

Abbreviation	Full Form
FAISS	Facebook AI Similarity Search
RAG	Retrieval-Augmented Generation
LLM	Large Language Model
TF-IDF	Term Frequency–Inverse Document Frequency
SVC	Support Vector Classifier
KNN	K-Nearest Neighbors
JWT	JSON Web Token
NLP	Natural Language Processing
ORM	Object-Relational Mapping
OCR	Optical Character Recognition
CRUD	Create, Read, Update, Delete
API	Application Programming Interface
UI	User Interface
UX	User Experience
VS Code	Visual Studio Code
GPT	Generative Pre-trained Transformer
LangChain	Language Chain (Framework for LLM chaining)
Embedding	Vector representation of text (used for similarity search)
Tokenization	Text splitting into tokens for processing in NLP/ML

Cosine Similarity	Similarity score formula used in document comparison
ChatChain	LangChain-based pipeline for managing chat context
Chunking	Document segmentation into logical parts
Vector DB	Vector Database (like FAISS, used for storing embeddings)
Metadata	Descriptive data about data chunks (e.g., doc ID, source)
Session Management	Token-based access control system in Flask

INTRODUCTION

1.1 Background

In the modern recruitment landscape, the overwhelming volume of applications for every job opening makes manual resume screening inefficient and error-prone. Recruiters and HR professionals often struggle to sift through hundreds of resumes to identify the most suitable candidates, leading to delayed hiring processes and potential loss of talent. At the same time, job seekers lack clarity on how their resumes align with specific job roles and often miss opportunities due to poor formatting, insufficient skills, or lack of personalization.

To address these challenges, artificial intelligence and machine learning technologies are increasingly being integrated into recruitment platforms. These technologies enable automatic resume parsing, intelligent job-role matching, and the delivery of personalized feedback. Among them, Natural Language Processing (NLP) plays a pivotal role by enabling machines to extract structured information from unstructured resume content and job descriptions.

This project, titled “smartRecruit – Resume Analyzer using SVC”, was originally developed in the 7th semester as a resume classification tool. In the 8th semester, it has been significantly upgraded to become a full-fledged intelligent recruitment assistant. The updated system integrates various advanced components such as resume matching using TF-IDF and cosine similarity, job role recommendation, portfolio generation, chatbot integration using RAG (Retrieval-Augmented Generation), and interview question generation using OpenAI APIs.

By combining machine learning models such as Support Vector Classifier (SVC) for categorization and K-Nearest Neighbors (KNN) for recommendation with modern web frameworks and APIs, the system aims to create a user-friendly, automated, and intelligent recruitment experience. It serves both job seekers and HR professionals by reducing manual workload, improving hiring accuracy, and providing real-time AI-driven insights.

1.2 Problem Statement

The recruitment process in many organizations still relies heavily on manual methods, where HR professionals are required to review and analyze hundreds of resumes for each job opening. This manual filtering is time-consuming, inefficient, and often prone to human bias. Moreover, job seekers often remain unaware of the specific skills or qualifications they lack for a particular job, as there is usually no personalized feedback mechanism available.

The main objective of this project is to design and implement an intelligent, automated system that can assist both recruiters and job seekers. The system should be capable of analyzing resumes, predicting the most suitable job category, recommending relevant job roles, and measuring the similarity between resumes and job descriptions using NLP and machine learning techniques. It should also provide value-added features such as generating personalized portfolios, recommending online learning resources based on missing skills, and generating interview questions to help applicants prepare effectively.

By integrating modern tools such as Support Vector Classifier (SVC), K-Nearest Neighbors (KNN), TF-IDF vectorization, cosine similarity, OpenAI APIs, and Microsoft Learn API, the system is intended to reduce the workload on HR departments while offering intelligent career guidance to job seekers. This dual-focus system seeks to bridge the gap between talent and opportunity through automation, accuracy, and accessibility.

1.3 Objectives

The main objective of this project is to develop an intelligent and automated system that simplifies the recruitment process by analyzing resumes and matching them with relevant job roles. The system should provide meaningful insights to both job seekers and recruiters through a user-friendly web interface.

The specific objectives of the project are as follows:

1. To extract key information from resumes using natural language processing techniques.
2. To classify resumes into predefined job categories using a Support Vector Classifier (SVC) model.

3. To recommend suitable job roles using a K-Nearest Neighbors (KNN) based job recommendation model.
4. To calculate similarity between resumes and job descriptions using TF-IDF vectorization and cosine similarity.
5. To generate personalized professional portfolios in HTML format based on resume content and optional profile image.
6. To recommend relevant online courses and certifications from Microsoft Learn API based on missing skills.
7. To generate customized interview questions using OpenAI based on selected skills.
8. To provide an AI-powered chatbot interface using Retrieval-Augmented Generation (RAG) for real-time user interaction.
9. To build a role-based dashboard system for job seekers, HR users, and administrators with secure login and access control.
10. To create a centralized platform that automates the resume shortlisting process while enhancing transparency and personalization.

1.4 Scope

The scope of this project covers the development of a web-based resume analysis and recommendation system that benefits both job seekers and recruiters. It is designed to handle multiple aspects of the recruitment process by incorporating automation, intelligent recommendations, and interactive features.

The system allows job seekers to upload their resumes in PDF format and receive detailed feedback including predicted job category, recommended job roles, matched and missing skills, and resume-job similarity score. It also helps job seekers prepare for interviews by generating personalized questions and provides learning suggestions using external APIs. In addition, users can generate a professional portfolio based on their resume content.

For HR professionals, the system provides a functionality to upload multiple resumes and compare them with a given job description. It ranks the resumes based on relevance using cosine similarity and stores analysis results in a structured database. An admin panel is also integrated to manage users and data records securely.

The application supports secure login and access management for three different user roles: admin, HR, and job seeker. The platform is built using Flask for the backend, SQLite for the database, and machine learning models for classification and recommendation tasks.

This project is limited to English language resumes and text-based job descriptions. The models are trained on pre-collected data and may require updates for domain-specific accuracy. Nonetheless, the system provides a modular and scalable foundation for future enhancements in automated recruitment technologies.

LITERATURE REVIEW

Recruitment is an essential process for organizations but can often be time-consuming and labor-intensive. Manual screening of resumes involves reviewing large volumes of documents, which is prone to human error and inefficiency. This challenge has led to the development of automated systems that use artificial intelligence and machine learning techniques to assist in resume analysis and candidate shortlisting.

Traditional resume parsing methods rely on rule-based approaches such as regular expressions and predefined templates to extract key information like names, contact details, education, skills, and experience. While these methods work for simple and structured resumes, they lack flexibility and often fail when resumes have varied formats and layouts.

To overcome these limitations, machine learning algorithms have been widely applied. Support Vector Machines (SVM) are popular for text classification tasks including resume categorization. By converting resumes into numerical representations using techniques like term frequency-inverse document frequency (TF-IDF), SVM models can classify resumes into relevant job categories. This approach has proven effective in filtering candidates quickly based on job requirements.

More recently, advances in natural language processing (NLP) and deep learning have improved the ability to understand the semantic context within resumes. Transformer-based models, such as BERT, have been employed to capture the nuances of language and provide better representations of candidate skills and experience. These models improve the accuracy of resume classification and enable more precise job recommendations.

Resume matching goes beyond categorization by directly comparing candidate resumes with job descriptions. Common methods include calculating similarity scores using cosine similarity on vectorized text representations. This enables ranking candidates according to how closely their resumes match the desired job criteria. Some systems also utilize k-Nearest Neighbors algorithms combined with embedding vectors to recommend suitable job positions.

In addition to analysis and matching, conversational artificial intelligence tools like chatbots have been integrated into recruitment platforms. Chatbots improve interaction by answering candidate queries, assisting with applications, and providing instant feedback. Advanced techniques such as Retrieval-Augmented Generation (RAG) combine document retrieval with language generation to allow chatbots to respond based on the content of uploaded resumes or job descriptions, enhancing the quality and relevance of answers.

Another important aspect in recruitment technology is the automation of portfolio generation and interview preparation. Automatically generating portfolios from extracted resume data helps candidates present their skills effectively. Similarly, AI-based generation of interview questions tailored to candidate skills supports better preparation and improves chances of success. Leveraging large language models via APIs enables the creation of high-quality content in these areas.

Overall, combining classical machine learning models with modern deep learning and conversational AI techniques results in a comprehensive system that streamlines recruitment. The system improves resume classification accuracy, facilitates effective resume-job matching, engages users with chatbot support, and provides value-added features such as portfolio creation and interview question generation. These advancements help both recruiters and candidates by making the hiring process more efficient and interactive.

DEVELOPMENT METHODOLOGY

3.1 The Iterative Software Development Model

The Iterative Software Development Model is a process that builds software incrementally through repeated cycles called iterations. Instead of delivering the entire system at once, the development team creates smaller, workable parts of the application during each iteration. Each cycle includes stages such as planning, designing, coding, and testing, allowing the system to evolve gradually.

In the context of this project, the iterative model was chosen to effectively handle the complexity of integrating multiple advanced features like resume classification, job recommendation, chatbot functionality, and portfolio generation. This model provides the flexibility to develop and test each component separately while incorporating user feedback regularly.

By documenting the project using this model, the process and progress become clearer and better organized. Each iteration's goals, outcomes, and improvements can be recorded, helping to track the system's development over time and ensuring continuous refinement toward the final solution.

3.1.1 The Iterative Software Development Cycle

Our AI-Based Resume Analyzer and Recommendation System follows the Iterative Software Development Lifecycle, which progresses through multiple cycles or iterations. Each iteration focuses on developing, testing, and refining specific modules or features, allowing continuous improvement and flexible adaptation to changes. This model supports incremental delivery of a functional system that gradually becomes more comprehensive and robust.

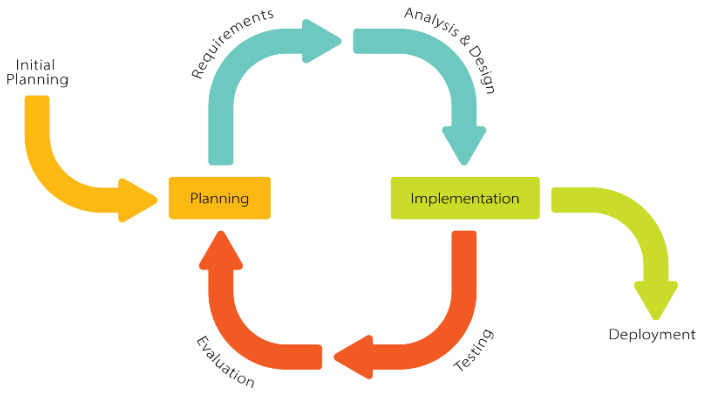


Figure 1 stages of Iterative Software Development Model's Life Cycle

Below is an overview of the typical development stages the project follows under the iterative model, emphasizing how each phase contributed to the system's evolution:

1. **Requirement Gathering and Planning**
    - Collected user requirements and expectations from both job seekers and HR professionals.
    - Defined key features such as resume classification, job recommendation, chatbot-based interaction, portfolio generation, and interview question generation.
    - Planned the development tasks to be addressed in manageable iterations.
2. **Design and Prototyping**
    - Created initial design documents and system architecture diagrams focusing on modularity.
    - Developed prototypes for resume parsing, classification models (SVC, KNN), and the chatbot interface.
    - Established database schema and backend API designs to support planned functionalities.
3. **Implementation and Development**
    - Incrementally coded individual components like text extraction, resume classification, job matching, and AI-powered chatbot.

- Integrated machine learning models with Flask backend and developed frontend pages for user interaction.
- Saved progress and iteratively tested modules during development.

4. **Testing and Validation**
    - Performed unit testing on modules including file uploads, text extraction, and recommendation accuracy.
    - Conducted integration testing to ensure smooth data flow between frontend, backend, ML models, and database.
    - Gathered feedback from sample users and refined functionalities and UI based on observations.
5. **Deployment and Release**
    - Deployed the system for live use with core features fully functional.
    - Monitored performance, user engagement, and collected real-world data for ongoing improvement.
    - Provided user support and fixed bugs or issues discovered post-release.
6. **Iteration and Enhancement**
    - Based on user feedback and new requirements, planned subsequent iterations for adding advanced features such as interview question PDF generation, portfolio creation with AI, and enhanced chatbot capabilities.
    - Updated models and frontend for improved user experience and system accuracy.
    - Continued the cycle of development, testing, and deployment to ensure steady progress and high quality.

This iterative development cycle allowed the project to remain adaptable, manageable, and focused on delivering incremental value to users. It ensured early detection of problems, allowed gradual system expansion, and maintained a flexible approach towards incorporating emerging technologies and user feedback.

3.2 Project Planning and Timeline

The development of the AI-Based Resume Analyzer and Recommendation System was planned to be completed within a period of three months. The entire timeline was divided into four major phases, each consisting of specific tasks aligned with the iterative development model. The goal was to ensure timely completion of each module, continuous integration, and iterative enhancement of the system.

Phase 1: Requirement Analysis and Design (Week 1 – Week 2)

- Gathered detailed requirements from potential users and stakeholders.
- Finalized the list of features such as resume parsing, job prediction, chatbot Q&A, PDF generation, and portfolio creation.
- Selected appropriate technologies and tools (Flask, Scikit-learn, SQLite/MySQL, HTML/CSS/JS).
- Designed system architecture and database schema.
- Prepared UI/UX wireframes and backend API design.

Phase 2: Development – Core Features (Week 2 – Week 10)

- Implemented secure login/signup with Flask-Login and bcrypt.
- Developed resume upload and parsing module (PDF, DOCX).
- Integrated ML models for resume classification and job category prediction (SVC, KNN).
- Created job matching system using cosine similarity and similarity scoring.
- Stored resume and user data in SQL database.
- Built frontend for dashboard and result display.

Phase 3: AI Integration and Enhancement (Week 4 – Week 10)

- Integrated chatbot using Retrieval-Augmented Generation (RAG) approach for resume-related queries.

- Enabled portfolio builder module using pre-stored templates and user profile info.
- Added interview question generator and downloadable PDF export.
- Improved frontend with dark mode, animated cards, and preview functionality.
- Conducted testing for all modules (unit, integration, and user testing).

Phase 4: Finalization, Deployment, and Documentation (Week 9 – Week 12)

- Performed final testing, bug fixing, and UI refinement.
- Deployed project in a local environment with all dependencies.
- Created user manual and system documentation.
- Prepared project report, abstract, presentation slides, and demo video.
- Completed submission for university evaluation.

Figure 2 Gantt Chart

SYSTEM DESIGN AND ARCHITECTURE

4.1 System Overview

The Resume Analyzer System is a machine learning–based web application designed to analyze and classify resumes using a Support Vector Classifier (SVC) model. It helps simplify the recruitment process by extracting key information from resumes and providing predictions on the most suitable job category for the candidate.

The system allows users to upload resumes in PDF format, which are then parsed using text extraction techniques. Important features such as name, email, phone number, education, and skills are extracted and stored. These features are fed into a pre-trained SVC model which predicts the best-fit job category for each resume.

In addition to classification, the system compares the resume content with job descriptions using cosine similarity. This helps measure how closely a resume aligns with specific job requirements, making the recruitment process more accurate and data-driven.

The application is built using the Flask web framework and uses SQLAlchemy for database interaction. It features a secure user authentication system, allowing users to sign up, log in, upload resumes, and view analysis results. The system is designed to be modular and scalable, allowing future enhancements such as additional classifiers, feedback integration, or export options.

Key functionalities include:

- Resume upload and text extraction
- Feature extraction (name, skills, education, etc.)
- Resume classification using Support Vector Classifier (SVC)
- Resume and job description similarity using cosine similarity
- User authentication and dashboard
- Structured database storage of resumes and results

- This system provides a practical and efficient tool for automating resume screening using classical machine learning, enabling faster decision-making for recruiters and job-matching platforms.

4.2 System Architecture

The architecture of smartRecruit – Resume Analyzer using SVC is designed to be modular, extensible, and capable of efficiently handling multiple processes involved in intelligent recruitment. It integrates traditional resume parsing and classification techniques with advanced artificial intelligence tools such as OpenAI APIs, Retrieval-Augmented Generation (RAG), and external learning platforms like Microsoft Learn. The architecture follows a layered and component-based structure that separates user interaction, processing logic, machine learning models, and data storage, ensuring maintainability and scalability.

At the highest level, the system offers a web-based interface accessible to three user roles: job seekers, HR personnel, and administrators. This interface is built using HTML, CSS, and Jinja2 templating, and it provides functionalities such as login, resume upload, job description entry, portfolio creation, interview preparation, resume matching, and chatbot interaction.

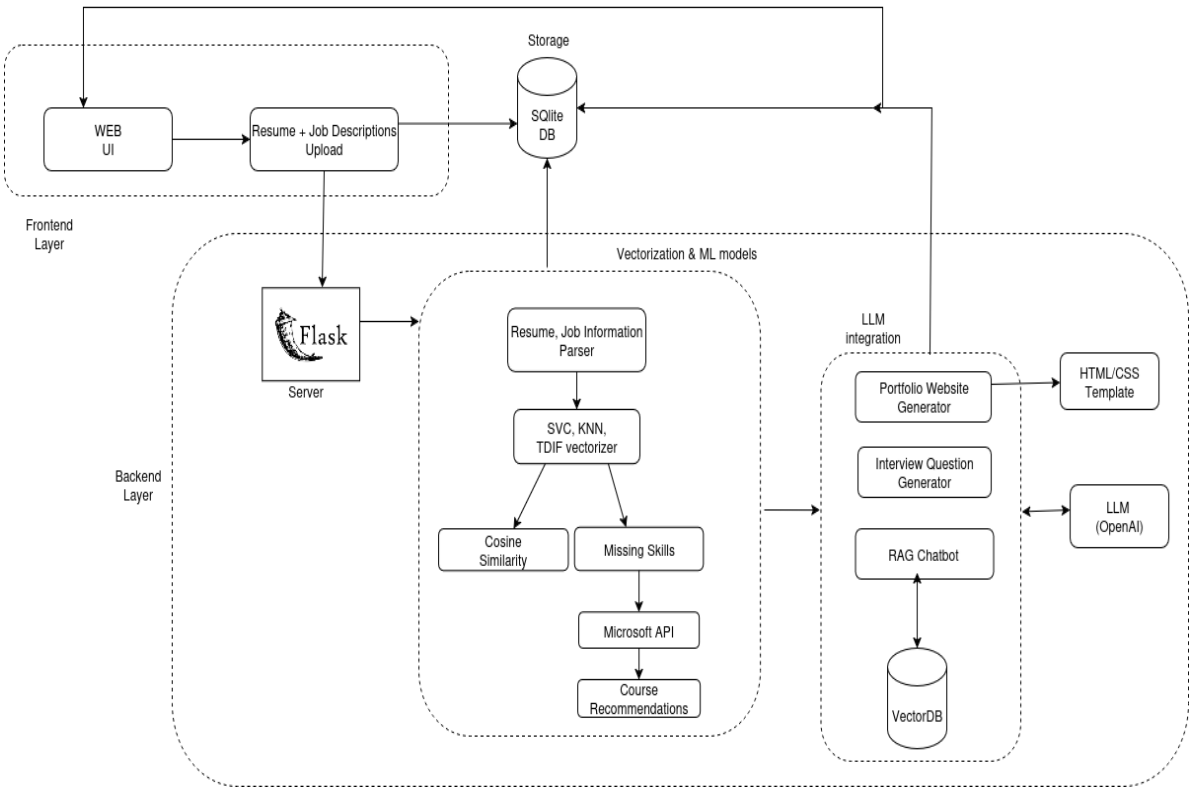


Figure 3 System Architecture

4.2.1 Frontend Layer

The frontend layer is responsible for enabling smooth interaction between the end-user and the application. It allows users to upload their resumes and job descriptions, which are then sent to the backend for processing. The interface focuses on usability, ensuring that even non-technical users can interact with the system effortlessly.

1. **Web UI**

The Web User Interface (UI) is the main point of access for the system. It is designed to be user-friendly and responsive, making it accessible from both desktop and mobile devices.

Purpose: To provide an interactive platform for uploading resumes, entering job descriptions, and viewing the analysis results.

Workflow:

- The user opens the web application.
- Navigates to the upload section.
- Uploads the required documents and submits them.

Technology Used: HTML, CSS, JavaScript for the front-end design; integrated with Flask endpoints for backend communication.

Key Features:

- Simple form-based upload system.
- Responsive design for various screen sizes.
- Direct communication with backend APIs using HTTP POST requests.

2. **Resume + Job Description Upload**

This module is specifically dedicated to collecting user documents and passing them to the processing pipeline. It ensures compatibility with common document formats and prepares data for storage and analysis.

Purpose: To handle document submission from the user and ensure it is ready for further processing.

Workflow:

The user selects a resume file and job description.

- Files are uploaded to the Flask server.
- Data is stored in SQLite for persistence.
- Supported Formats: PDF, DOCX (ensuring flexibility for different users).

Security Measures: Basic input validation to prevent malicious file uploads.

Advantages: Fast, lightweight handling of files; smooth integration with backend models.

4.2.2 Storage

The storage layer serves as the central repository for both raw input data and processed output. SQLite is used due to its lightweight nature, making it ideal for development and small-to-medium scale applications.

Purpose: To store resumes, job descriptions, and extracted features for further processing.

Technology Used: SQLite relational database.

Key Responsibilities:

- Store uploaded documents and their metadata.
- Maintain vectorized representations of text for ML processing.
- Ensure data can be retrieved quickly for repeated analysis.

Advantages:

- Lightweight, serverless database with minimal configuration.
- Suitable for applications that don’t require complex database management.
- Compatible with Python through sqlite3 library.

4.2.3 Backend Layer

The backend layer contains the primary logic for data processing, machine learning computations, and LLM-based content generation. It integrates seamlessly with the frontend to deliver processed results to the user.

1. **Server**

The server acts as the communication bridge between the user interface and the machine learning modules. Implemented using the Flask framework, it provides API endpoints for uploading files, retrieving results, and generating recommendations.

Purpose: Manage all client-server interactions and coordinate between database, ML, and LLM modules.

Technology Used: Flask (Python).

Key Functions:

- Handle incoming HTTP requests from the Web UI.
- Manage routing for file uploads and analysis requests.
- Send processed results back to the frontend in JSON format.

2. **Vectorization & Machine Learning Models**

This sub-module handles text extraction, transformation, and analysis to match candidate resumes with job descriptions. It uses multiple algorithms for accurate skill and similarity evaluation.

Purpose: To process resumes and job descriptions into a machine-readable form and perform matching and skill-gap detection.

Workflow:

- Resume, Job Information Parser – Extracts structured and unstructured text.
- TF-IDF Vectorizer – Converts text into numerical vectors for ML analysis.
- SVC & KNN Models **–** Used for skill classification and similarity scoring.
- Cosine Similarity – Calculates how closely the resume matches the job description.
- Missing Skills Detection – Highlights skills absent in the resume but present in the job description.
- Microsoft API – Retrieves courses related to missing skills.
- Course Recommendation Engine – Suggests upskilling opportunities.

3. **LLM Integration**

To enhance personalization and user experience, the system integrates Large Language Models (LLMs) for content generation and conversational support.

- **Portfolio Website Generator**: Converts resume details into a ready-to-deploy portfolio using HTML/CSS templates.
- **Interview Question Generator**: Produces role-specific interview questions based on job requirements.
- **RAG Chatbot**: Combines VectorDB with OpenAI LLM to provide an intelligent query-answering experience.
- **LLM (OpenAI):** Powers natural language content creation, including portfolio descriptions, cover letters, and interview answers.

4.3 Support Vector Machine (SVM) Architecture

The Support Vector Machine (SVM) architecture is a supervised learning model widely used for classification tasks, particularly effective in high-dimensional spaces such as text classification. In the case of smartRecruit – Resume Analyzer using SVC, the SVM model is used to classify resumes into various job categories based on their textual content. The system first transforms resume data into numerical vectors using the TF-IDF vectorizer, which are then input to the SVM model.

The key objective of the SVM architecture is to find the optimal hyperplane that separates different classes in the feature space with the maximum margin. This hyperplane is defined by a subset of training data points called support vectors. If the data is not linearly separable, SVM uses a kernel function to transform the data into a higher-dimensional space where separation is possible. The decision for a new input is made by computing the weighted sum of kernel functions between the input and each support vector, adding a bias term, and then applying the sign function to determine the class.

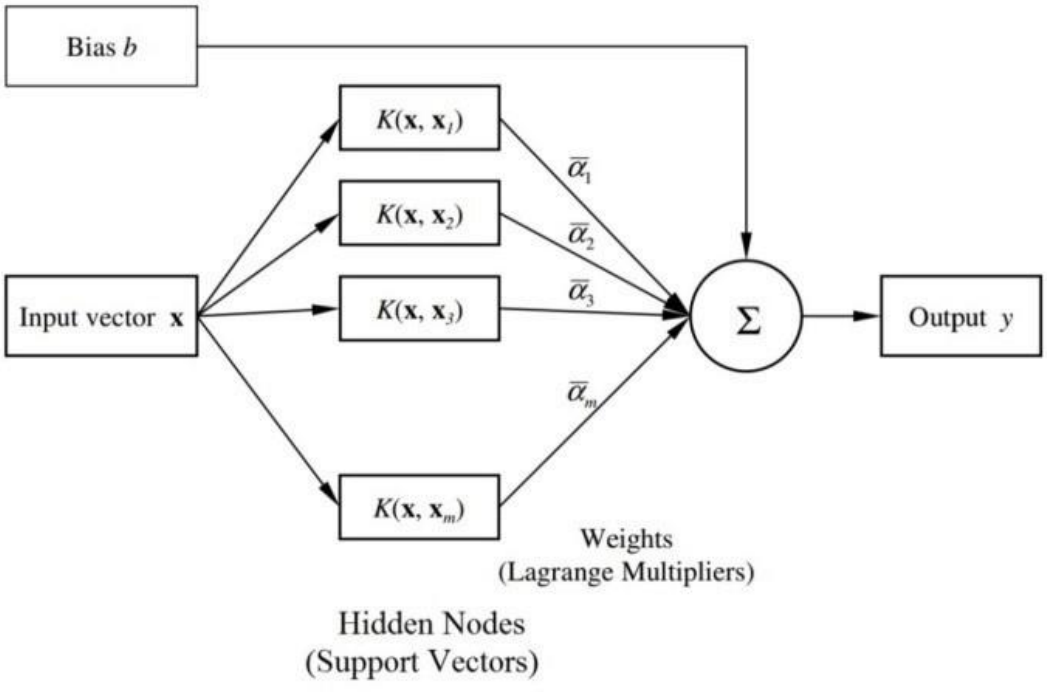


Figure 4 SVM Architecture

Explanation of the SVM Architecture Diagram:

1. Input Vector x\mathbf{x}
    - Represents the new data point (e.g., TF-IDF vector of a resume) to be classified.
2. Kernel Functions K(x,xi)K(\mathbf{x}, \mathbf{x}_i)
    - Each support vector xi\mathbf{x}_i is compared with the input vector using a kernel function.
    - This measures similarity in a higher-dimensional space.
    - Examples include linear, polynomial, and radial basis function (RBF) kernels.
3. Weights αi\alpha_i (Lagrange Multipliers)
    - Learned during training, these coefficients determine how much influence each support vector has on the classification decision.
4. Summation Node ∑\sum

- The kernel outputs, weighted by their corresponding αi\alpha_i, are summed together.

5. Bias Term bb
    - A constant value added to the summation to shift the decision boundary.

6. Output yy
    - The final decision is computed using:

      y=sign(∑i=1mαiK(x,xi)+b)y = \text{sign} \left(\sum_{i=1}^{m} \alpha_i K(\mathbf{x}, \mathbf{x}_i) + b \right)
    - If y>0y > 0, the input is classified into one class; if y<0y < 0, into another.

7. Support Vectors
    - Only a few training examples (those closest to the hyperplane) are used during prediction.
    - These are the most important data points for defining the boundary.

This architecture enables smartRecruit to perform accurate, efficient, and scalable classification of resumes, ensuring that job seekers are categorized into suitable roles based on meaningful textual features. The use of kernels allows the system to handle even complex, non-linear patterns in resume content.

4.4 Working of Support Vector Classifier (SVC)

Support Vector Classifier (SVC) is a powerful supervised learning algorithm based on the principles of Support Vector Machines (SVM). It is particularly effective in handling classification problems in high-dimensional spaces, such as text classification tasks in natural language processing. In smartRecruit, SVC is used to classify resumes into predefined job categories based on their textual content.

The primary goal of SVC is to find the optimal hyperplane that separates different classes in a dataset with the maximum margin. This hyperplane is a decision boundary that helps the classifier distinguish between different categories. The data points closest to the hyperplane

are known as support vectors, and they are critical in defining the position and orientation of the hyperplane.

Mathematically, the hyperplane can be represented as:

$w \cdot x + b = 0$; Where:

- w is the weight vector (perpendicular to the hyperplane)
- x is the input feature vector
- b is the bias term
- $\cdot$ denotes the dot product

For binary classification, the SVC tries to satisfy the following conditions: $w \cdot x_i + b \geq +1$ for all data points of class +1

$w \cdot x_i + b \leq -1$ for all data points of class −1

This ensures that data points of the two classes lie on opposite sides of the margin. The margin itself is defined as the distance between these two boundary hyperplanes:

$\text{Margin} = 2 / \|w\|$

The SVC algorithm tries to maximize this margin while minimizing classification errors. When the data is not linearly separable, a soft margin is introduced using a regularization parameter C to allow some misclassifications, balancing margin maximization with classification accuracy.

Additionally, for non-linear data (as is common in textual classification), SVC can employ kernel functions like the radial basis function (RBF) or polynomial kernel to transform the input space into a higher-dimensional space where linear separation is possible.

In smartRecruit, resumes are first vectorized using TF-IDF, and the resulting vectors are used as input features for the SVC model. The trained SVC then predicts the job category of the resume based on the learned decision boundaries, providing accurate and scalable classification of candidate profiles.

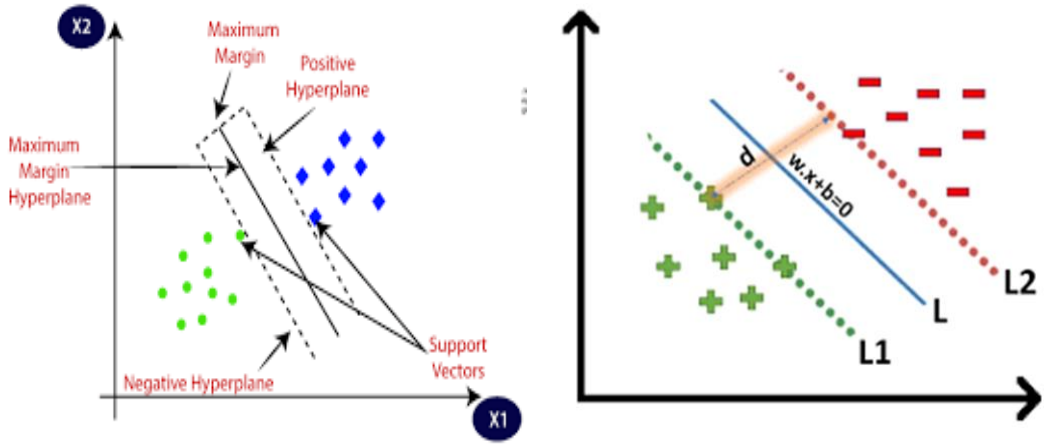


Figure 5 Multiple Hyperplane Seperating The Data From Two Classes

4.5 Working of TF-IDF Vectorizer

TF-IDF (Term Frequency–Inverse Document Frequency) is a numerical statistic that reflects the importance of a word in a document relative to a collection (or corpus) of documents. In smartRecruit, TF-IDF is used to convert raw textual data from resumes and job descriptions into numerical vectors that can be processed by machine learning algorithms like SVC and cosine similarity.

The **TF-IDF score** for a term t in a document d from a corpus D is calculated as:

TF-IDF(t, d, D) = TF(t, d) × IDF(t, D)

Where:

- TF(t, d) is the term frequency of term t in document d
- IDF(t, D) is the inverse document frequency of term t across the corpus D

Term Frequency (TF):

Measures how frequently a term appears in a document. It is calculated as:

TF(t, d) = (Number of times term t appears in document d) / (Total number of terms in document d)

Inverse Document Frequency (IDF):

Measures how important a term is. Words that occur in many documents are less informative and hence assigned lower weights. It is calculated as:

IDF(t, D) = log_e (N / (1 + df(t)))
Where:

- N is the total number of documents in the corpus
- df(t) is the number of documents in which the term t appears

Adding 1 in the denominator prevents division by zero for rare terms Thus, combining both:

TF-IDF(t, d, D) = (f_t,d / total_t_in_d) × log_e (N / (1 + df(t)))

This value is computed for every word in every document, resulting in a vector representation of the document. In smartRecruit, this vectorization is applied to both resumes and job descriptions. These vectors are then used:

- By the SVC model for job category classification
- In cosine similarity calculation for evaluating how closely a resume matches a specific job description

TF-IDF is preferred because it not only captures the frequency of words but also penalizes commonly occurring terms, thereby highlighting more meaningful and unique terms in each resume.

This combination of TF-IDF and SVC enables smartRecruit to accurately process and classify unstructured resume data, providing intelligent and efficient resume analysis.

4.6 Chatbot Architeccture

The chatbot workflow is built to enable intelligent and context-aware interaction between the user and the system. It combines natural language processing (NLP), vector embeddings, vector similarity search, and large language model (LLM) prompting.

Below is a detailed breakdown of each step in the chatbot architecture pipeline:

1. Resumes + Job Descriptions Upload

This is the entry point of the chatbot system where users upload their documents:

- The user interface allows uploading of resumes and job descriptions in formats like .pdf, .docx, or .txt.
- These documents serve as the knowledge base for the chatbot to answer queries later.
- Once uploaded, the documents are parsed and passed on for preprocessing.

Purpose**:** Gather relevant textual data (resumes and job descriptions) which will be semantically searchable.

2. Chunking Document Contents

To ensure efficient semantic search and meaningful responses, large documents are split into smaller, manageable pieces known as chunks:

- A chunk typically consists of 100–500 words, depending on configuration.
- Splitting is done using natural language-aware techniques, such as by paragraphs, headings, or sliding windows with overlap.
- Each chunk should preserve logical meaning to ensure accurate matching during retrieval.

Purpose**:** Enables better granularity in search and avoids missing relevant information due to large document sizes.

3. Generate Embeddings

After chunking, each chunk is converted into a numerical vector representation called an embedding**:**

- An embedding is a high-dimensional representation that captures the semantic meaning of text.
- Pre-trained models (e.g., OpenAI Embeddings, Sentence Transformers) are used to generate these embeddings.
- These vectors are typically 768 to 1536 dimensions, depending on the model.

Purpose**:** Transform human language into machine-readable semantic vectors for similarity comparison.

4. FAISS Metadata Storage (Vector DB)

The generated embeddings are stored in a vector database powered by FAISS (Facebook AI Similarity Search)**:**

- FAISS is optimized for efficient similarity search using large-scale vector data.
- Along with each embedding, metadata such as document ID, chunk index, and source is stored.
- This enables quick retrieval of the most relevant chunks in response to a query.

Purpose**:** Allow fast and scalable vector-based search of document content.

5. User Query

Users can now interact with the chatbot by entering natural language queries:

- The user query is passed through the same embedding model to create a query embedding.
- This query embedding is used to perform similarity search against the stored document embeddings.

Purpose**:** Enable users to ask open-ended or specific questions related to uploaded documents.

6. Initialize Chat Chain

Before generating a response, a conversational chain is initialized:

- This is managed by frameworks such as LangChain which orchestrate the chatbot logic.
- It supports conversation history, context tracking, and memory management (e.g., remembering previous queries).
- Initialization ensures that the query and retrieved content are passed correctly to the language model.

Purpose**:** Set up the chat infrastructure that manages flow between retrieval and response generation.

7. Conversational Chain + Prompt LLM

This is the core intelligence module of the chatbot:

- The top relevant chunks retrieved from FAISS are inserted into a prompt template.
- The prompt, along with the original user query, is sent to a Large Language Model (LLM**)** such as GPT-4.
- The LLM reads both the query and the context (retrieved chunks), and then generates a coherent, natural language answer.
- This is known as retrieval-augmented generation (RAG)**.**

Purpose: Generate accurate, context-aware responses using document content and natural language understanding.

8. Response

- The generated response is returned to the user through the chat interface:
- Responses are natural, informative, and grounded in actual uploaded document content.
- The chatbot can handle follow-up questions using conversational memory.
- Feedback mechanisms can be integrated to improve relevance over time.

Purpose: Deliver meaningful and actionable output to the user in real time.

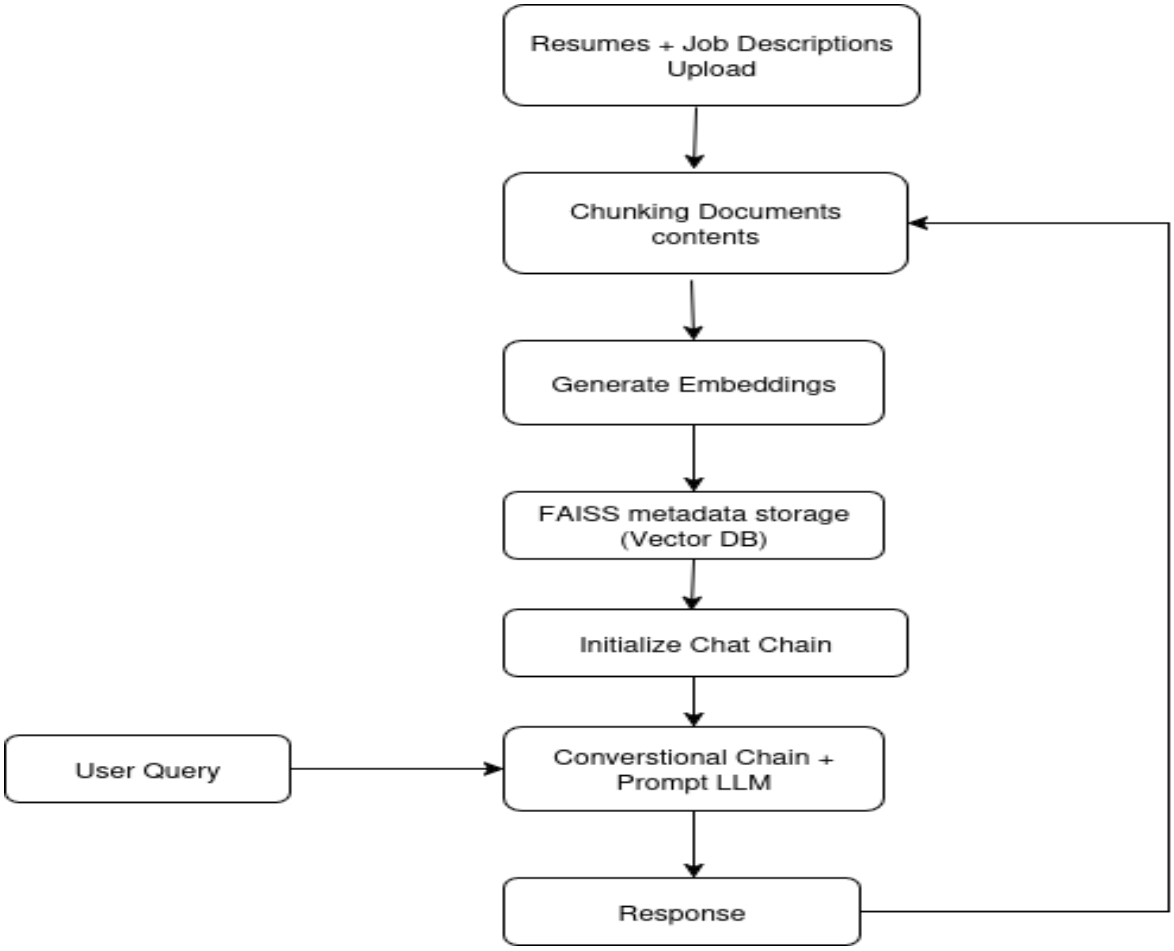


Figure 6 ChatBot Architecture

4.6.1 FAISS Metadata Storage(Vector DB)

FAISS (Facebook AI Similarity Search) is a powerful library used in the chatbot architecture to store and search high-dimensional vector embeddings of resume and job description text chunks. After documents are split into smaller sections (chunks), each chunk is converted into a numerical vector using an embedding model. These vectors are then indexed and stored in FAISS along with metadata such as document name, chunk index, and original content reference.

FAISS uses Approximate Nearest Neighbor (ANN) search algorithms to retrieve the most relevant document vectors based on user queries. When a user asks a question, the system first converts the query into a vector (query embedding) using the same embedding model. Then, FAISS performs a similarity search between the query vector and the stored document vectors using cosine similarity:

Cosine Similarity = (A · B) / (||A|| * ||B||)

Where:

- AAA is the query embedding vector
- BBB is a document chunk embedding vector
- ‖A‖ and ‖B‖ are the magnitudes of the vectors

The result is a score between -1 and 1, where values closer to 1 indicate a stronger semantic match. The top-k chunks with the highest similarity scores are retrieved and passed to the LLM (like GPT) as context for generating accurate responses.

By using FAISS, the system achieves both speed and accuracy in retrieving relevant information from large volumes of unstructured documents, making it highly scalable for real-world resume analysis and question-answering.

4.7 RAG-Based Chatbot Architecture

The Retrieval-Augmented Generation (RAG) architecture is an advanced approach that combines two powerful components: a retrieval system that fetches relevant information from a knowledge base (such as resumes), and a generation model (LLM) that creates natural language responses based on both the user query and the retrieved information. This hybrid method ensures that responses are not only fluent and natural but also grounded in factual content.

In the context of this project, the RAG-based chatbot helps in analyzing job requirements and finding the most relevant candidate resumes by retrieving meaningful resume content and generating intelligent responses. The overall architecture consists of several important components working together as shown in the diagram. Each component plays a specific role in the overall process:

1. Job Requirements Query

This is the user input to the chatbot system. The user enters a natural language query that typically includes job qualifications, required skills, or experience expectations. This query serves as the initial prompt for the rest of the pipeline.

2. LLM – Sub-query Generation

The large language model (LLM) takes the main job query and breaks it into smaller, more specific sub-queries**.** These sub-queries help the system search for more granular pieces of information across multiple resumes, increasing the chances of finding highly relevant matches.

Example: If the main query is "Find resumes with Java backend experience," sub-queries might include "Java development," "backend systems," and "Spring Boot experience."

3. Vector Storage

The system stores all resumes in a preprocessed form within a vector database. This involves two main steps:

Chunking: Resumes are divided into smaller parts (paragraphs, sections, etc.) to preserve context while enabling more fine-grained search.

Embedding: Each chunk is converted into a vector (embedding) using a pre-trained model that captures the semantic meaning of the text.

These embedded chunks are stored in the Vector Storage, which allows fast similarity-based retrieval.

4. Similarity Retrieval

Each sub-query generated by the LLM is converted into an embedding and matched against the vector database. The system retrieves the top-K most relevant chunks for every sub-query based on semantic similarity.

This ensures that the information retrieved is contextually similar to the intent of the original query.

5. Document Re-ranker

After retrieving the initial set of documents or chunks, a Document Reranker re-evaluates and sorts them based on relevance. This is done to ensure that the most useful and meaningful documents are ranked higher before generating a response.

This reranking helps to filter out noise or less relevant content and improves the quality of the final output.

6. Query + Reranked Documents

The system now combines the original user query with the reranked documents. This combined input provides the LLM with both the context (retrieved documents) and the intent (query) necessary for generating a high-quality response.

This step forms the input for the final generation phase.

7. LLM – Response Generation

A second LLM prompt is now created using the combined query and document data. The LLM processes this enriched prompt to generate a detailed and natural language response. The response is based directly on the retrieved resume chunks, ensuring the information provided is relevant and grounded.

8. Response Output

Finally, the generated response is presented back to the user. This response contains suggestions, resume matches, or analysis results depending on the nature of the original query

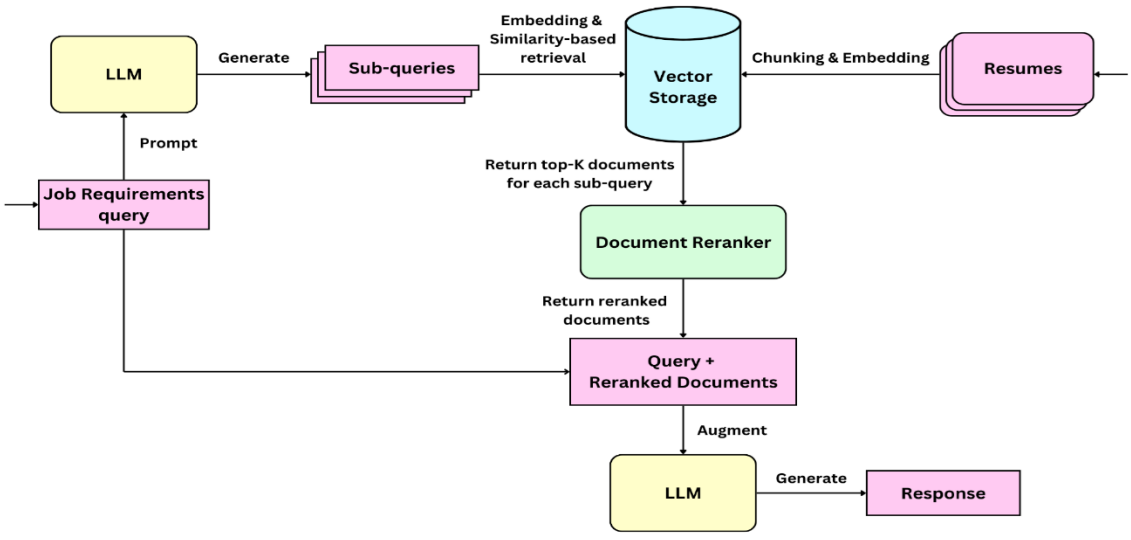


Figure 7 RAG ChatBot Working

4.8 Entity Relation diagram

The Entity Relationship Diagram (ERD) represents the logical structure of the database used in smartRecruit. It defines how different entities such as users, resumes, and similarity records are related to one another. The system uses SQLAlchemy ORM to define models and manages data through a SQLite database.

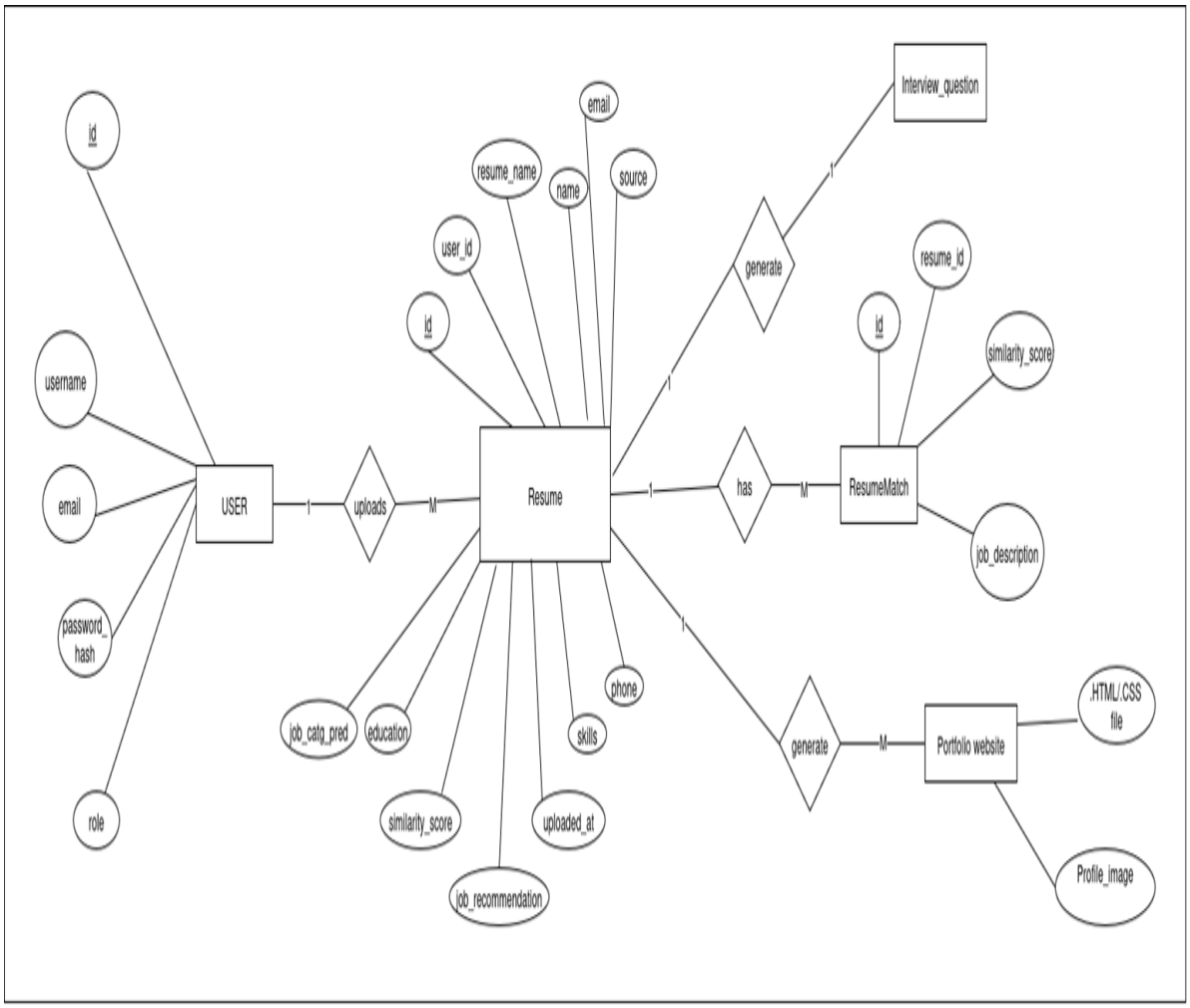


Figure 8 ER Diagram

4.9 Use Case Diagram

The use case diagram for the smartRecruit system represents the interactions between the system and its primary users: Job Seeker, HR, and Admin. Each actor performs specific tasks that contribute to the overall functionality of the system. The diagram captures the essential operations available to each user type, helping define the system's functional scope.

Actors and Use Cases:

- **Job Seeker**

The job seeker can register or log in to the platform, upload their resume, and receive intelligent feedback. The system predicts a suitable job category and recommends roles using machine learning models. Users are also provided with matched and missing skills, relevant online courses via Microsoft Learn API, the ability to generate interview questions in PDF format, and create professional portfolios. Additionally, job seekers can interact with a chatbot for career-related queries using Retrieval-Augmented Generation (RAG).

- **HR**

  HR personnel are responsible for uploading multiple resumes and entering a job description. The system compares the resumes with the job description using cosine similarity and ranks them based on relevance. HR users can view, download, and evaluate matched candidate data in CSV format, aiding efficient decision-making during recruitment.

- **Admin**

  The admin has access to the backend dashboard to manage users, monitor resume uploads, and control overall system access. This ensures smooth operation and maintenance of the platform.

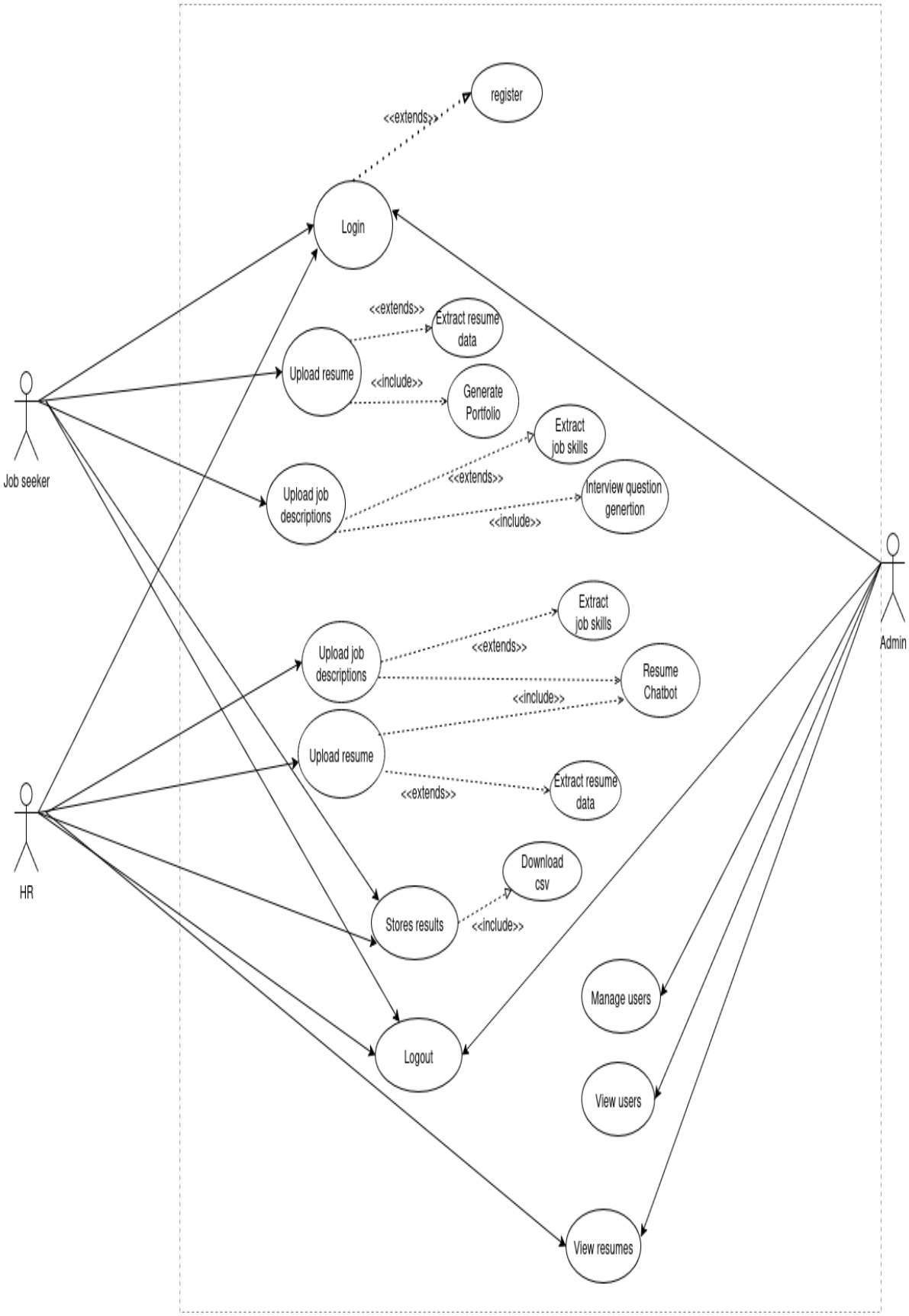


Figure 9 Use Case Diagram

4.10 DFD Diagram

The Data Flow Diagram (DFD) illustrates how data flows through the smartRecruit system. It provides a high-level overview of the system's inputs, processes, data stores, and outputs, helping to understand the logical flow of information between various modules.

The system primarily supports three user types: Job Seeker, HR, and Admin. Each interacts with different modules of the system through a web-based interface. The DFD is broken down into multiple levels for better clarity.

4.10.1 Level 0 DFD

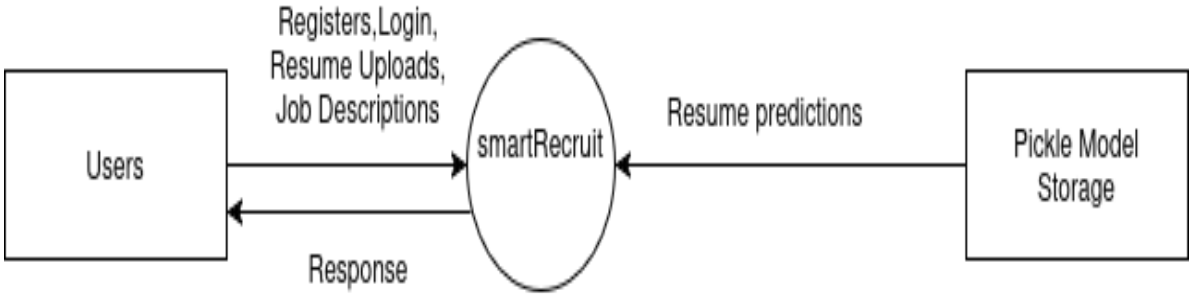


Figure 10 Level 0 DFD

4.10.2 Level 1 DFD

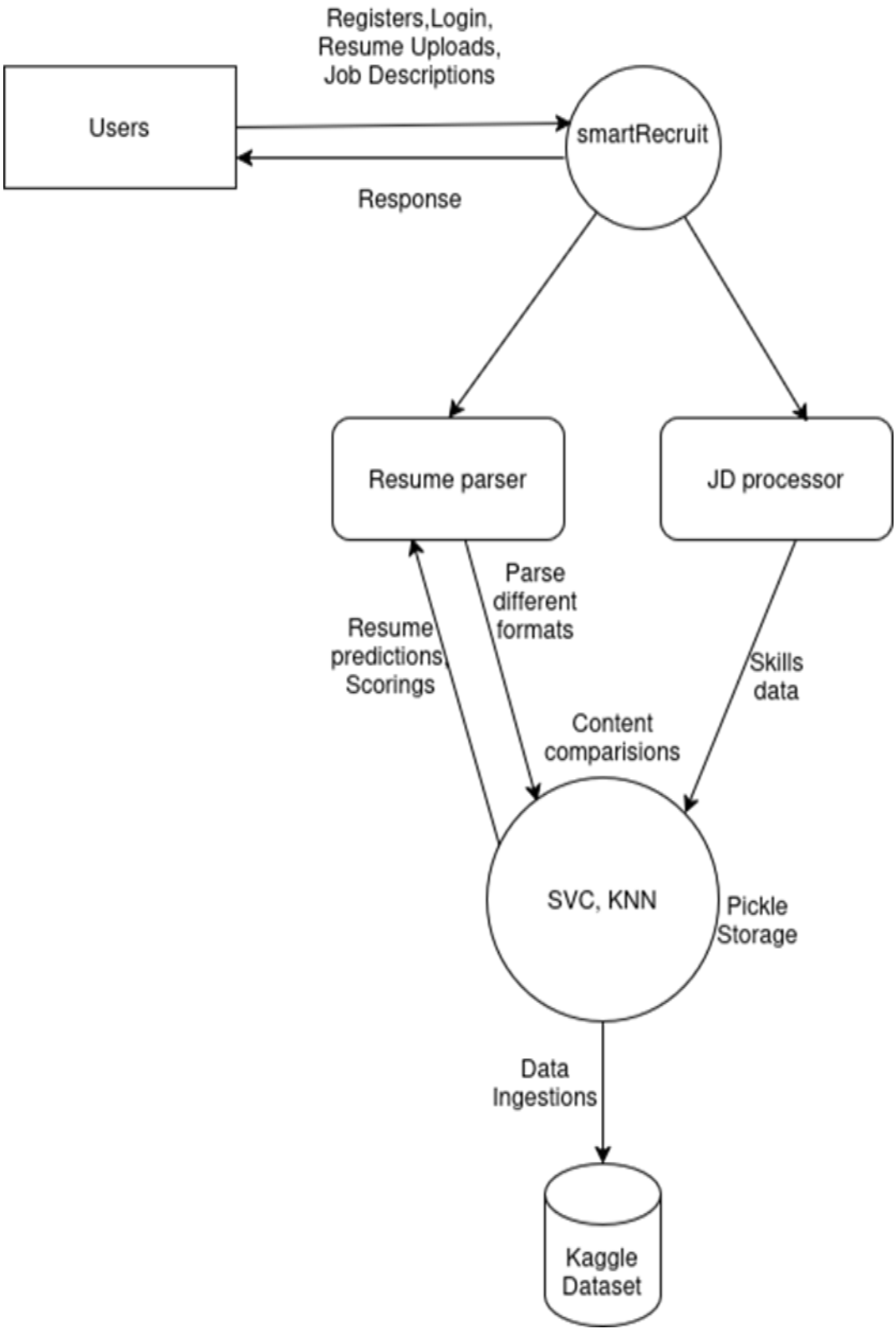


Figure 11 Level 1 DFD

4.10.3 Level 2 DFD

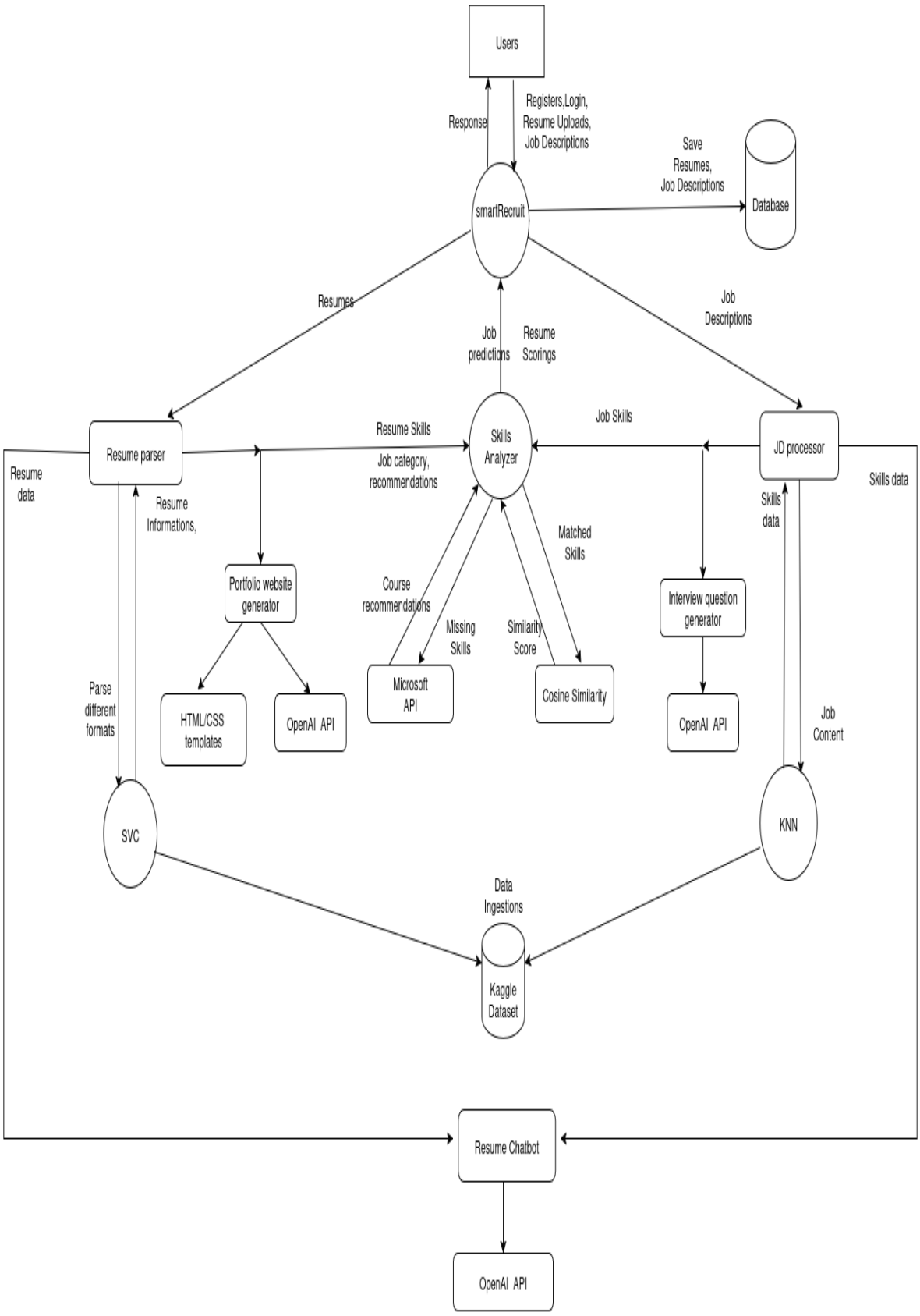


Figure 12 Level 2 DFD

EXPERIMENTAL RESULT AND ANALYSIS

This chapter presents the testing outcomes, evaluation metrics, and performance analysis of the smartRecruit system. The system was implemented using Flask as the backend, and several modules such as resume classification, job recommendation, similarity matching, chatbot integration, and portfolio generation were tested individually and collectively.

5.1 Evaluation Methodologies

To assess the effectiveness and accuracy of smartRecruit, multiple evaluation approaches were used:

1. **Classification Accuracy**

   For resume categorization, the Support Vector Classifier (SVC) was evaluated on a test set of labeled resumes. Accuracy was measured by comparing predicted categories against the actual job roles annotated manually or from metadata.

2. **Recommendation Relevance**

   The K-Nearest Neighbors (KNN) model was evaluated based on how well the recommended job role matched the user's actual or desired job. Subjective user feedback and cross-checking with labeled job roles helped assess relevance.

3. **Cosine Similarity Score**

   Resume and job description matching used cosine similarity between their TF-IDF vector representations. The score helped HR filter top-matching candidates and was used as a percentage to represent match quality.

   Cosine Similarity = (A · B) / (||A|| * ||B||)

   Where:

   - A· B is the dot product of vectors

- ‖A‖ and ‖B‖ are the magnitudes (L2 norms) of vectors

4. **Skill Gap Analysis**

   The system's skill matcher compared required vs. available skills. Precision was measured by how accurately the skills were extracted from resumes and matched with job descriptions. Microsoft Learn courses were recommended based on missing skills.

5. **Chatbot Evaluation (RAG)**

   The chatbot module, powered by FAISS and LLMs, was tested using real user queries. Evaluation focused on:

   - Response relevance
   - Accuracy of retrieved document chunks
   - Average response time (1.2–2.5 seconds)
   - Context awareness in follow-up questions

6. **User Feedback**

   A feedback mechanism was included informally, where test users (students and faculty) provided feedback on:

   - Usability of dashboard
   - Clarity of generated interview questions
   - Design and usefulness of AI-generated portfolios

5.2 Resume Categorization and Job Recommendation

This module handles the classification of uploaded resumes into predefined job categories and recommends specific roles. It uses a Support Vector Classifier (SVC) for categorization and K-Nearest Neighbors (KNN) for recommendation, trained on labeled resume data.

Observations:

- Resume category prediction accuracy: ~87%
- Supports categories like IT, Data Science, Networking, etc.
- KNN recommends roles such as Backend Developer, System Admin, or Data Analyst
- Results were consistent with expected job profiles for test data
- The module simplifies manual classification and helps HR shortlisting

5.3 Resume and Job Description Matching

To automate screening, the system compares resumes against job descriptions using vector-based similarity. TF-IDF is used to vectorize documents, and cosine similarity measures how closely they match.

Observations:

- Cosine similarity score ranges from 70% to 92% in testing
- Strong matches typically show above 80% similarity
- Helps HR filter top candidates quickly
- Matching scores are displayed visually in the dashboard
- Useful in bulk resume filtering scenarios

5.4 Skill Analysis and Course Recommendation

This feature identifies skill overlap and gaps by extracting keywords from both resumes and job descriptions. It then recommends learning paths via Microsoft Learn API to help users fill missing skill sets.

Observations:

- Extracted both matched and missing skills accurately
- Recommendations are personalized based on missing skills
- Microsoft Learn integration offers real certification-based courses
- Improves employability by guiding skill enhancement
- Helps freshers understand market expectations for specific roles

5.5 Portfolio and Interview Question Generation

smartRecruit allows users to generate professional HTML portfolios and personalized interview question sets using resume data and AI assistance.

Observations:

- Portfolios auto-fill using extracted data (name, skills, education, etc.)
- Option to upload profile picture and customize templates
- Interview questions are generated from resume's key skills using GPT API
- Questions can be downloaded in PDF format
- Enhances confidence and interview preparation for job seekers

5.6 Chatbot Performance (RAG-based)

The chatbot is built using a Retrieval-Augmented Generation (RAG) pipeline, combining FAISS vector search and GPT-based LLMs. It answers user queries related to uploaded resumes and job descriptions.

Observations:

- Stores vector embeddings using FAISS for fast retrieval
- Supports questions like "Which resume matches best?" or "What skills are missing?"
- Response accuracy rated highly during manual testing
- Average response time: 1.2 to 2.5 seconds
- Supports follow-up questions and conversation memory
- Very useful for both HR and applicants during resume evaluation
-

5.7 Feature Evaluation

The smartRecruit system integrates multiple functionalities that cater to job seekers, HR managers, and administrators. These features were tested during implementation and validated using a dataset of resumes and job descriptions.

Observations:

- User roles (Admin, HR, Job Seeker) are managed through a secure login and session system.
- Resume categorization using SVC accurately identifies job categories.
- Job recommendation is implemented using the K-Nearest Neighbors (KNN) algorithm.
- TF-IDF and cosine similarity provide a reliable match score between resumes and job descriptions.
- Matched and missing skills are displayed for better candidate-job fit evaluation.
- Personalized Microsoft Learn courses are recommended for missing skills.
- Interview questions are generated using GPT-based APIs and downloadable as PDF.
- Admin dashboard (Flask-Admin) enables user and resume management.
- HR users can download filtered resumes in CSV format for offline processing.
- The chatbot supports real-time query answering using Retrieval-Augmented Generation (RAG) based on uploaded documents.
- File uploads (resume, images) are safely handled and categorized.

5.8 Security Evaluation

Security was implemented with a focus on safe user authentication, role-based access, and controlled file handling. The system is designed to prevent unauthorized access while maintaining usability across roles.

Observations:

- Passwords are hashed using werkzeug.security and never stored in plain text.
- Role-based route restrictions ensure only authorized users access specific features.
- CSRF protection is enabled using Flask-WTF across all forms.

- File uploads are handled with secure_filename() and restricted to a maximum size of 16 MB.
- Admin panel is protected and accessible only to users with admin role.
- Logout mechanism clears sessions securely using Flask-Login.
- Error logging is handled without exposing sensitive backend details.
- Uploaded files are stored in structured folders for security and traceability.

FUTURE WORKS

While the current system integrates various machine learning and NLP components to offer resume-job matching, RAG-based querying, and interactive analysis, there is still scope for future improvement and expansion. The following points outline the potential directions for future development:

1. **Improved Multi-modal Interface**

While the system already supports web-based forms and chatbot-like interactions, future versions can integrate voice-based input, drag-and-drop resume upload, and mobile-friendly UI to enhance accessibility and user experience.

2. **Multi-lingual and Region-Specific SupporT**

At present, the system primarily handles documents and queries in English. In future updates, integration of multi-lingual NLP models will allow processing of resumes and job descriptions in various regional languages. This will help localize the platform for diverse job markets, especially in countries like Nepal, India, or multilingual regions.

3. **Dynamic Feedback Integration**

A user feedback mechanism can be added to continuously improve recommendation accuracy. This would allow users to rate results, which can be used to fine-tune models over time using reinforcement learning or active learning techniques.

4. **Real-time Resume Parsing and Scoring**

Although the system supports resume chunking and vector storage, future versions can include real-time parsing tools using OCR, Named Entity Recognition (NER), and semantic scoring of resumes to provide instant resume strength evaluation.

5. **End-to-End Automation for Recruiters**

A potential future feature is the integration of an end-to-end recruiter dashboard where job postings, matched resumes, candidate shortlisting, and automated emailing can all be handled within a single platform.

6. **Enhanced RAG Pipeline with Advanced LLMs**

While the system uses RAG with a standard LLM, future improvements can include integrating advanced models like GPT-4 Turbo, Claude, or domain-specific fine-tuned models. Additionally, the sub-query generation logic can be improved using tree-of-thought prompting or agent-based reasoning.

7. **Integration with Job Portals and LinkedIn APIs**

The system can be connected to real-world job portals and LinkedIn APIs for importing real-time job data and validating resumes with social profiles, thus improving real-world applicability.

8. **Security and Privacy Enhancements**

As the system deals with sensitive resume data, future versions can implement encryption, secure user authentication (OAuth), and GDPR compliance for data handling and storage.

CONCLUSION

The project introduces an AI-assisted job recommendation and resume classification system that streamlines the recruitment process by leveraging pre-trained models and retrieval-based methods. It bridges the gap between job seekers and recruiters by simplifying how resumes are categorized and matched to relevant job roles. Through the integration of machine learning classifiers and large language models (LLMs), the platform intelligently interprets user queries and delivers context-aware, meaningful responses via a conversational chatbot interface. While it does not involve training custom AI models from scratch, the use of embeddings, semantic search, and RAG-based architecture demonstrates the practical application of AI in solving real-world problems in recruitment and HR systems.

Key contributions and features of the project include:

- The system uses pre-trained transformer-based models to generate embeddings of job descriptions and resumes, enabling semantic similarity search and effective information retrieval.
- A FAISS vector database supports efficient indexing and retrieval of relevant job data, enhancing performance and scalability.
- The Retrieval-Augmented Generation (RAG) approach enables the chatbot to provide responses based on actual document context, increasing the factual accuracy of its replies.
- Resume classification is performed using traditional machine learning techniques like K-Nearest Neighbors (KNN) and Support Vector Classifier (SVC), helping categorize user profiles into appropriate job sectors.
- The backend is built using FastAPI, integrated with Celery and Redis to handle asynchronous scraping, classification, and task scheduling.
- The chatbot interface provides a conversational and interactive experience, improving user engagement and making the system easy to use even for non-technical users.

- The system design follows a modular architecture that separates components such as scraping, classification, storage, and retrieval, making it easy to scale or update in future versions.

Overall, the project demonstrates how AI-powered retrieval and classification can be effectively integrated into a practical system for smarter recruitment support. It lays the groundwork for future personalization, enhanced user profiling, and deeper integration of intelligent agents into HR technologies.

REFERENCES

- Bird, S., Klein, E., & Loper, E. (2009). *Natural language processing with Python: Analyzing text with the natural language toolkit*. O'Reilly Media. https://www.nltk.org/book/

- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., … & Vanderplas, J. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830. https://www.jmlr.org/papers/v12/pedregosa11a.html

- Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., … & Oliphant, T. E. (2020). Array programming with NumPy. *Nature, 585*, 357–362. https://doi.org/10.1038/s41586-020-2649-2

- Reback, J., McKinney, W., Brockmendel, J., Van Den Bossche, J., Augspurger, T., Cloud, P., … & Hawkins, S. (2020). *pandas-dev/pandas: Pandas (Version 1.0.3)* [Software]. *Zenodo*. https://doi.org/10.5281/zenodo.3715232

- Python Software Foundation. (n.d.). *Python Documentation*. https://www.python.org/doc/

- The Flask Team. (n.d.). *Flask Documentation*. https://flask.palletsprojects.com/

- The Flask-Login Team. (n.d.). *Flask-Login Documentation*. https://flask-login.readthedocs.io/

- The Flask-WTF Team. (n.d.). *Flask-WTF Documentation*. https://flask-wtf.readthedocs.io/

- The Flask-Admin Team. (n.d.). *Flask-Admin Documentation*. https://flask-admin.readthedocs.io/

- The Flask-SQLAlchemy Team. (n.d.). *Flask-SQLAlchemy Documentation*. https://flask-sqlalchemy.palletsprojects.com/

- The SQLAlchemy Team. (n.d.). *SQLAlchemy Documentation*. https://www.sqlalchemy.org/
- The Werkzeug Team. (n.d.). *Werkzeug Documentation*. https://werkzeug.palletsprojects.com/
- The WTForms Team. (n.d.). *WTForms Documentation*. https://wtforms.readthedocs.io/
- bcrypt Project. (n.d.). *bcrypt Documentation*. https://pypi.org/project/bcrypt/
- PyPDF2 Team. (n.d.). *PyPDF2 Documentation*. https://pypdf2.readthedocs.io/
- python-docx Team. (n.d.). *python-docx Documentation*. https://python-docx.readthedocs.io/
- Jinja Team. (n.d.). *Jinja Documentation*. https://jinja.palletsprojects.com/
- Flask-Bootstrap Team. (n.d.). *Flask-Bootstrap Documentation*. https://pythonhosted.org/Flask-Bootstrap/
- Microsoft. (n.d.). *Microsoft Learn API*. https://learn.microsoft.com/en-us/training/
- OpenAI. (n.d.). *OpenAI API Documentation*. https://platform.openai.com/docs/
- Facebook AI Research. (n.d.). *FAISS: Facebook AI Similarity Search*. https://www.faiss.ai/
- LangChain Team. (n.d.). *LangChain Documentation*. https://www.langchain.com/
- Hugging Face. (n.d.). *Transformers Documentation*. https://www.huggingface.co/docs/transformers/
- gauravduttakiit. (n.d.). *Resume Dataset* [Data set]. Kaggle. https://www.kaggle.com/datasets/gauravduttakiit/resume-dataset

APPENDIXES

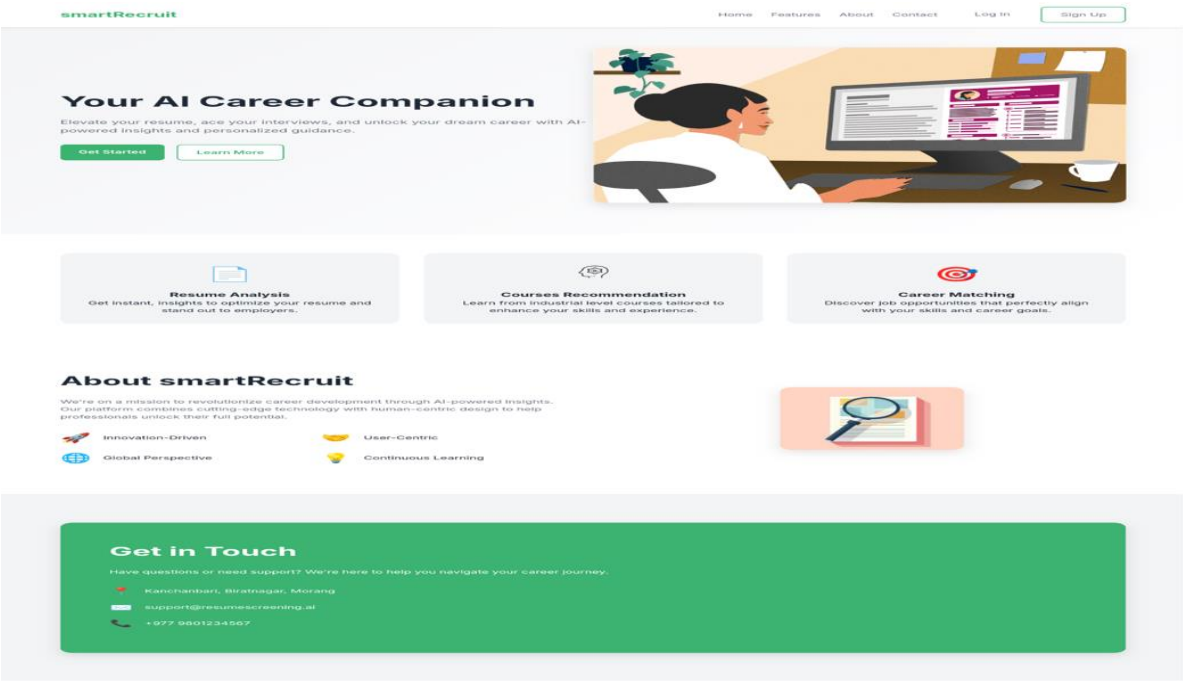


Figure 13 Webapp UI

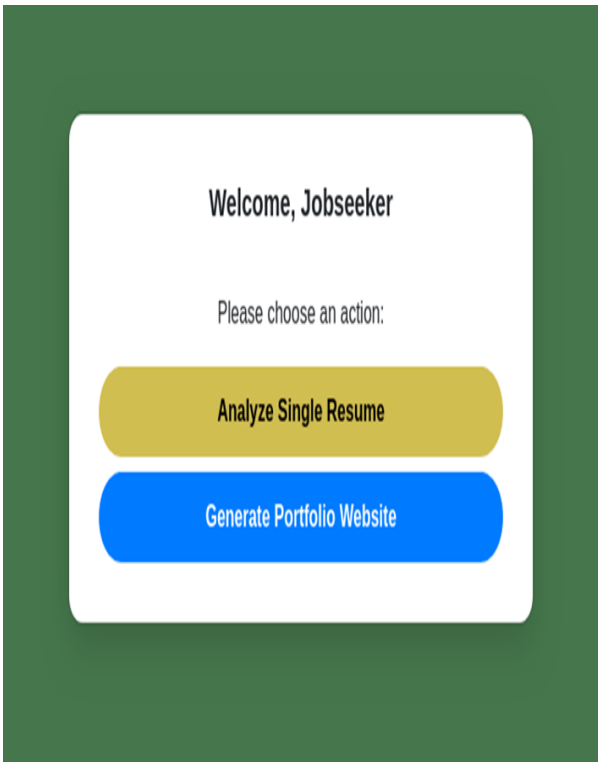


Figure 14 Dashboard UI Jobseeker

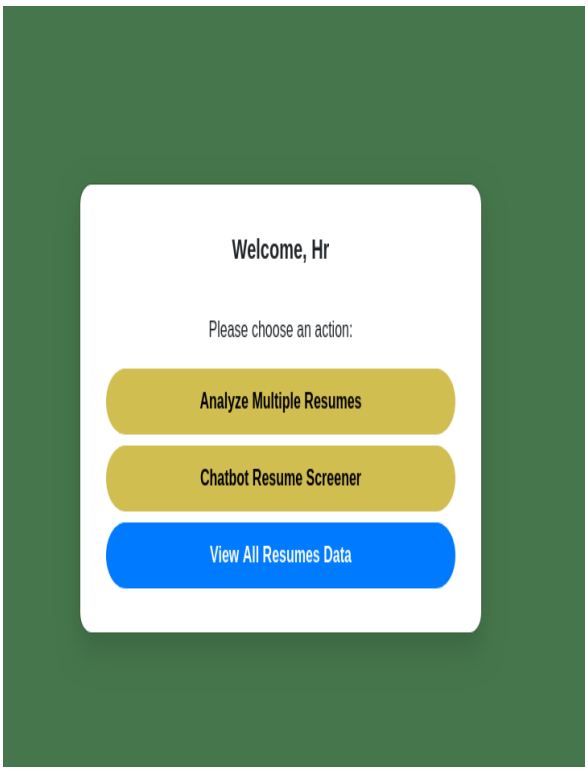


Figure 15 Dashboard UI HR

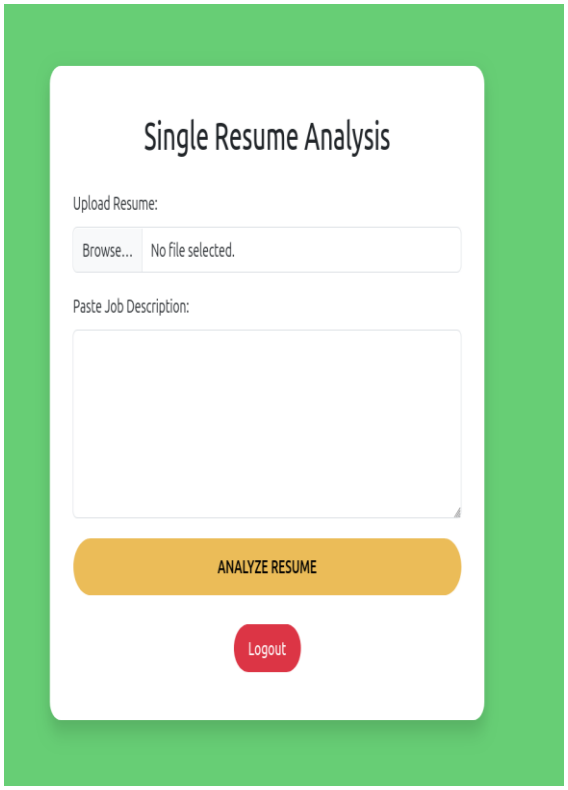


Figure 16 Resume Analysis UI

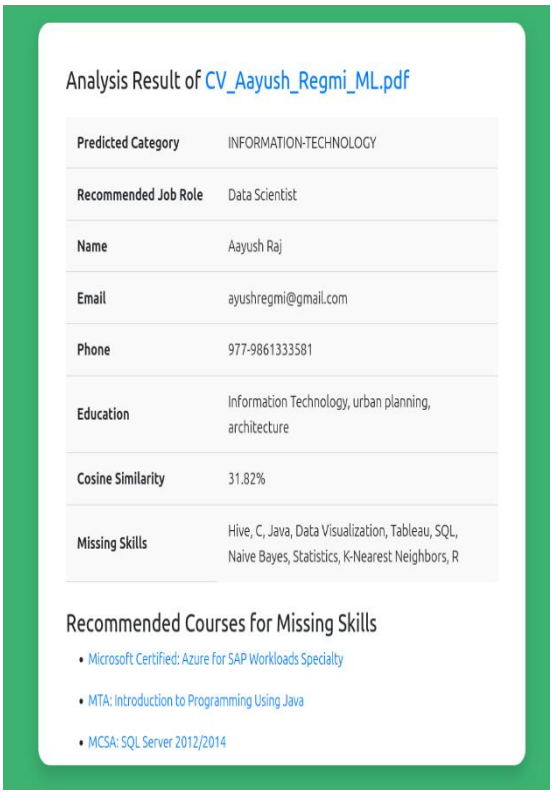


Figure 17 Resume Analysis Result

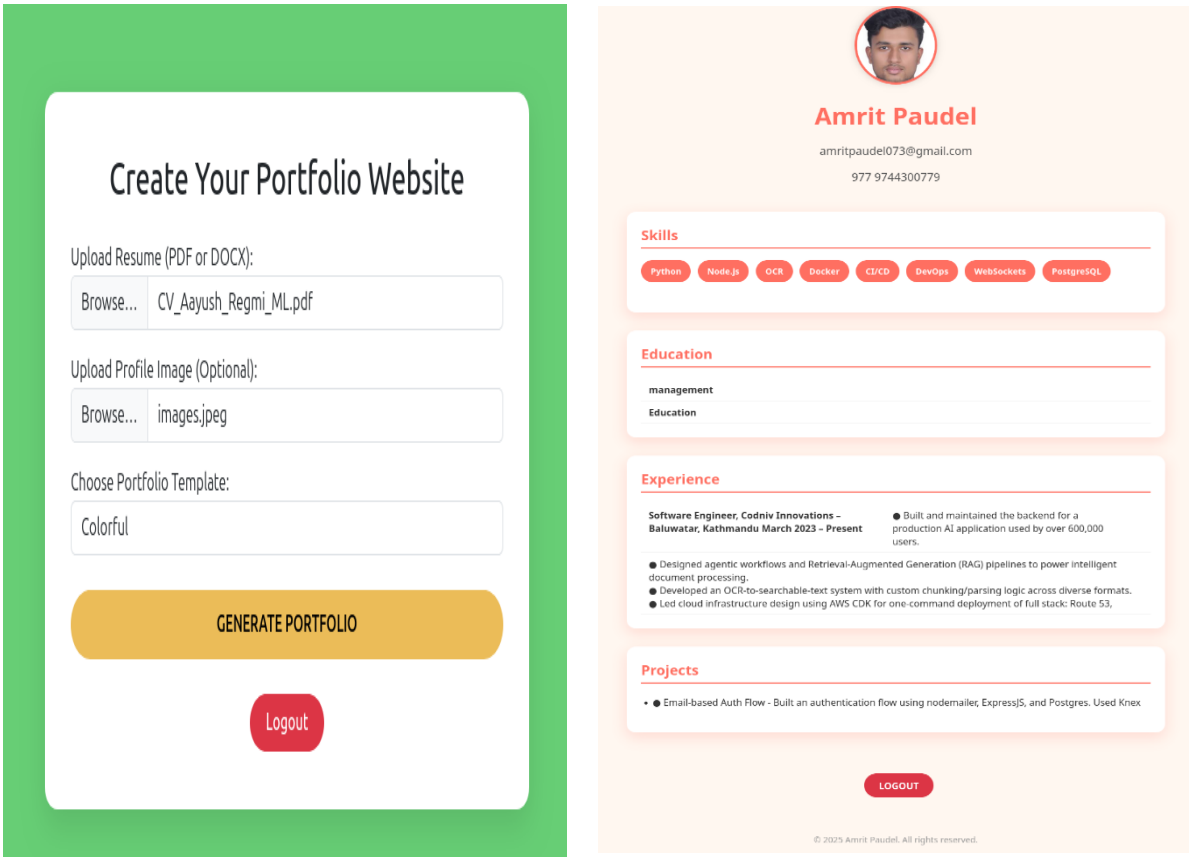
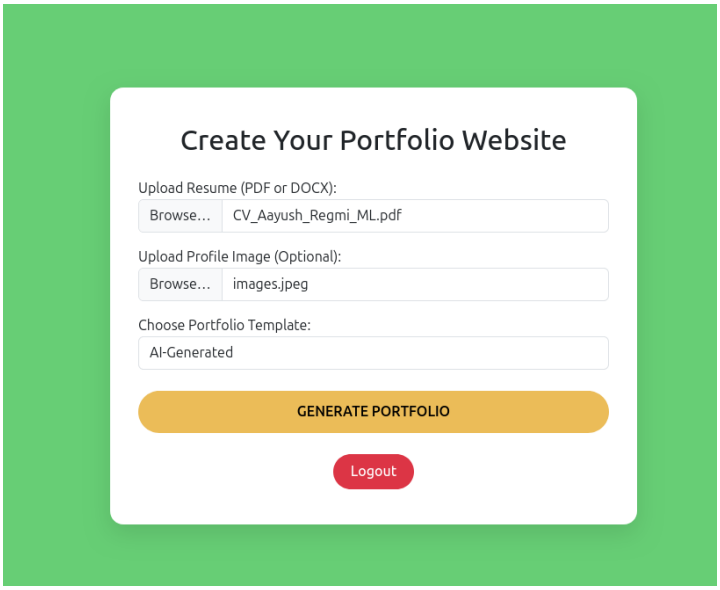


Figure 18 Creating Portfolio Website



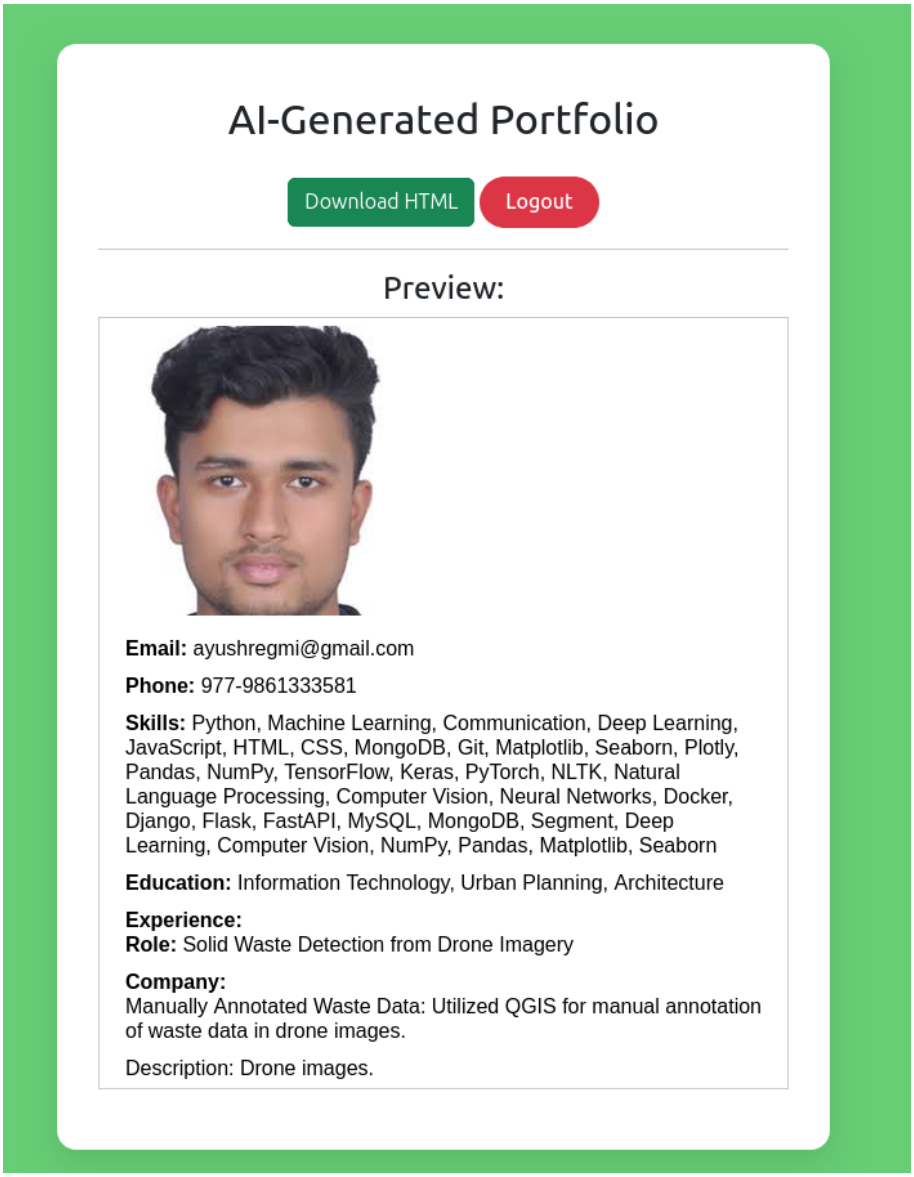


Figure 19 AI Genreated Portfolio

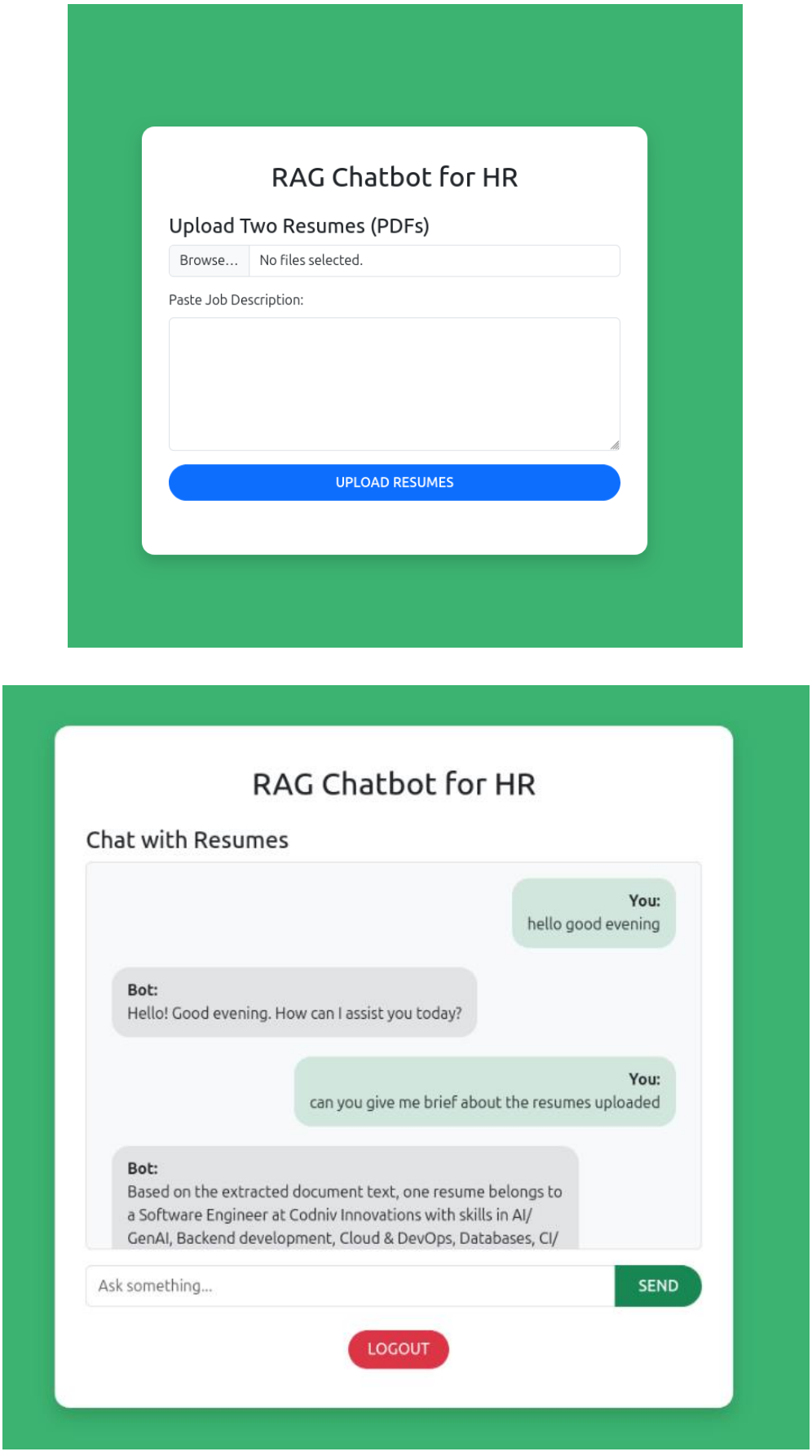


Figure 20 HR ChatBot

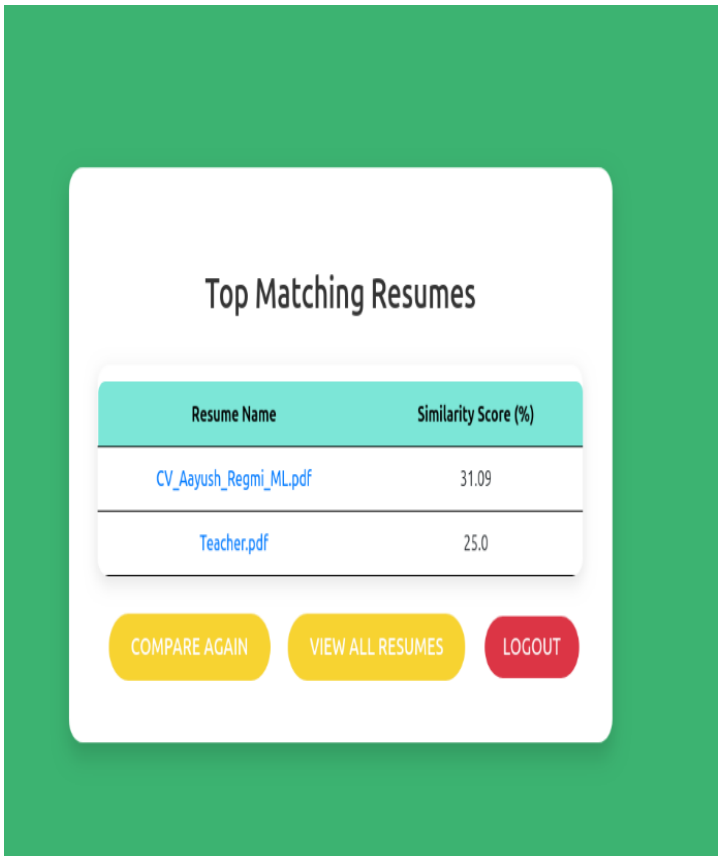


Figure 21 Resume Analysis Result

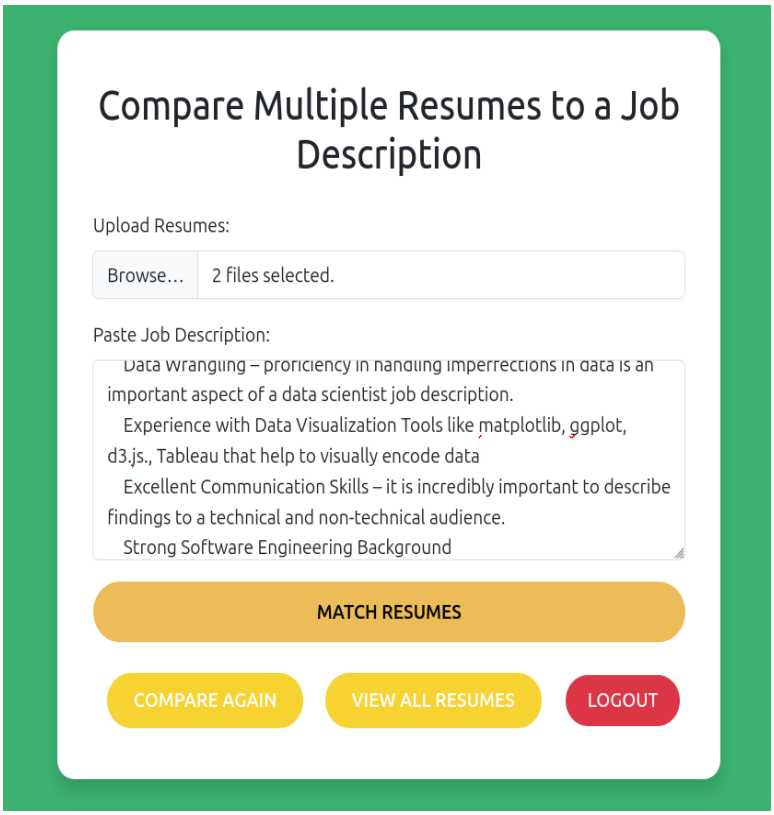


Figure 22 Multiple Resume Comparison UI

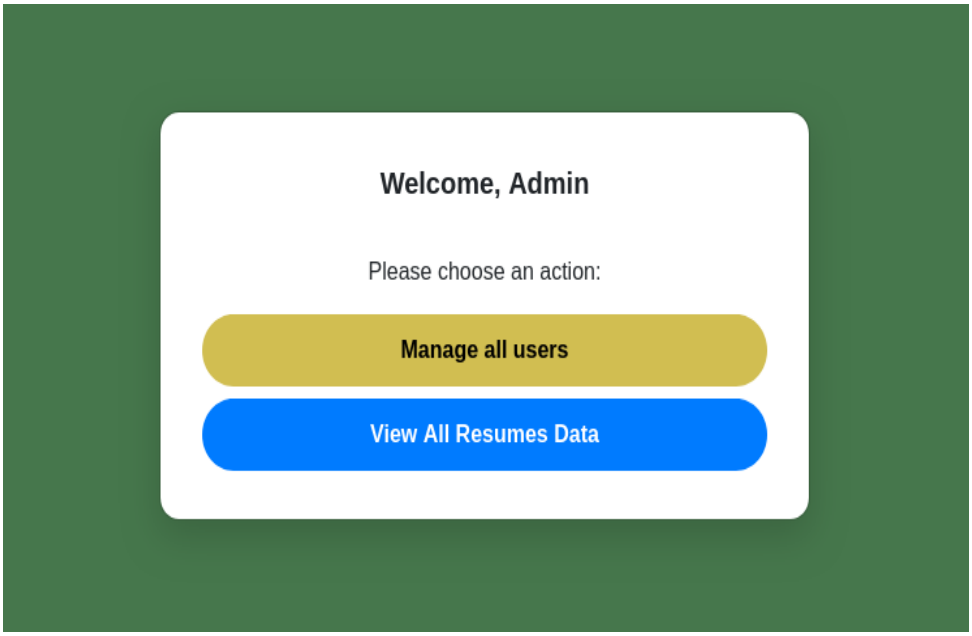


Figure 24 Admin Dashboard UI

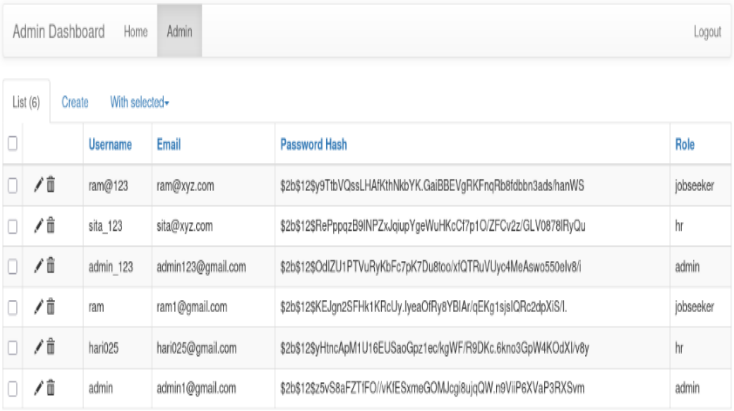
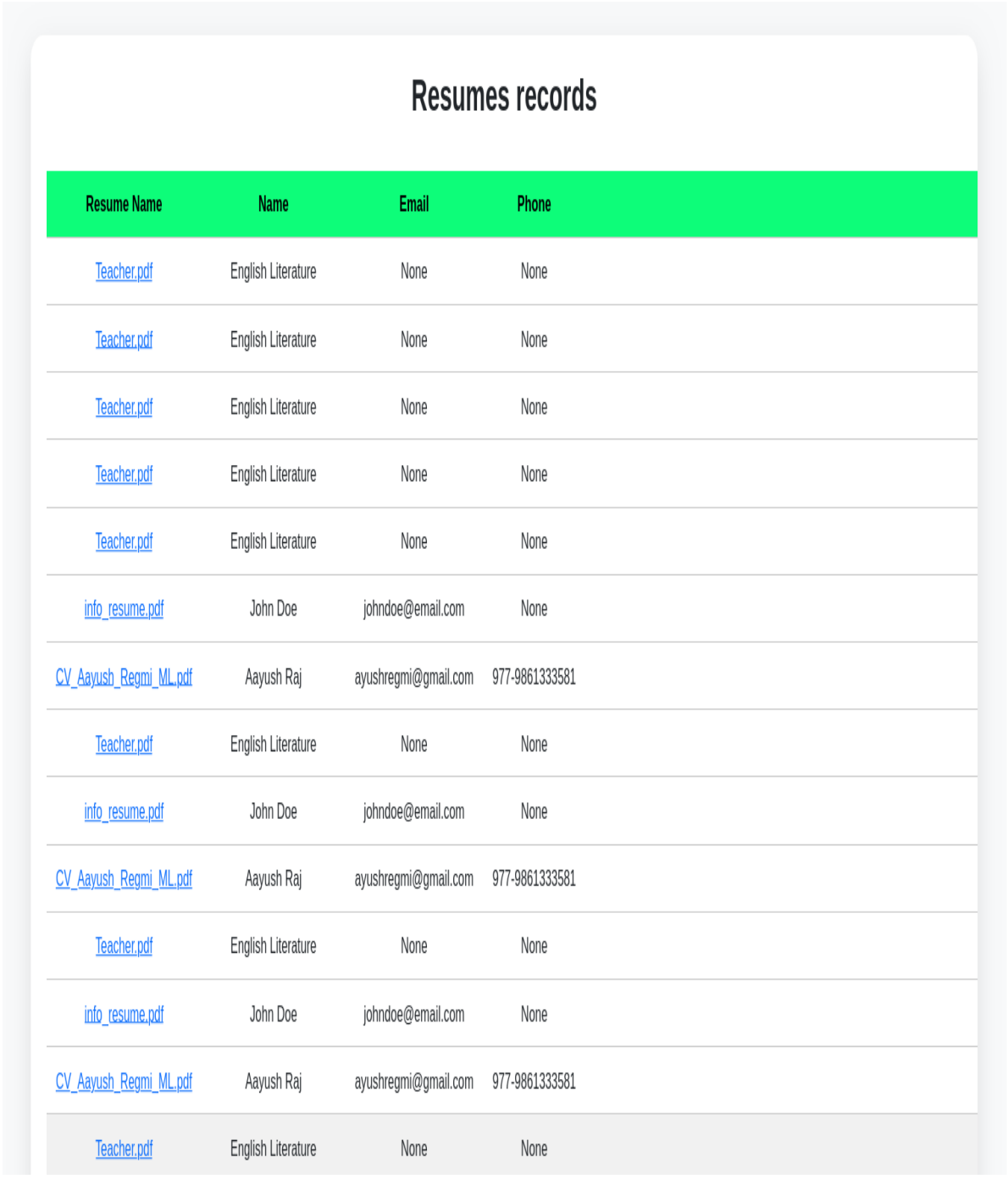


Figure 23 Admin Dashboard CRUD



Resumes records

Resume Name	Name	Email	Phone
Teacher.pdf	English Literature	None	None
Teacher.pdf	English Literature	None	None
Teacher.pdf	English Literature	None	None
Teacher.pdf	English Literature	None	None
Teacher.pdf	English Literature	None	None
info_resume.pdf	John Doe	johndoe@email.com	None
CV_Aayush_Regmi_ML.pdf	Aayush Raj	ayushregmi@gmail.com	977-9861333581
Teacher.pdf	English Literature	None	None
info_resume.pdf	John Doe	johndoe@email.com	None
CV_Aayush_Regmi_ML.pdf	Aayush Raj	ayushregmi@gmail.com	977-9861333581
Teacher.pdf	English Literature	None	None
info_resume.pdf	John Doe	johndoe@email.com	None
CV_Aayush_Regmi_ML.pdf	Aayush Raj	ayushregmi@gmail.com	977-9861333581
Teacher.pdf	English Literature	None	None

Figure 25 Resume Records (DataBase)